

Intent-Driven Orchestration for Scientific Computing

Jessica Dagostini

May 2026

Contents

1	Introduction	1
1.1	Prior Work	4
1.2	Contributions and Proposal	6
2	Background	8
2.1	High Performance and Scientific Computing	8
2.1.1	Resource Allocation and Scheduling	9
2.1.2	Performance Portability and Application Profiling	11
2.2	Intent-based systems	12
3	Completed Research	15
3.1	miniGiraffe: A Pangenome Proxy App	15
3.1.1	Summary	19
3.2	A Toolbox for Load Balancing Simulation for AMReX Applications	20
3.2.1	Summary	22
4	Proposed Research	23
4.1	User-Aware Intent-Based System	23
4.1.1	Cluster resampling	28
4.1.2	Surrogate Runtime Model	29
4.1.3	Intent-Driven Configuration Selection	31
4.2	Cluster-aware intent-driven system	36
5	Conclusion	40
A	miniGiraffe information	49

Abstract

The growing complexity of high-performance computing (HPC) systems poses a fundamental challenge for domain scientists, whose primary objective is to obtain scientifically valid results rather than to optimize resource utilization. Modern leadership-class facilities combine heterogeneous CPUs, GPUs, and specialized accelerators across systems that simultaneously support traditional scientific simulations and AI-driven workloads. This creates a vast, machine-dependent configuration space that even experienced systems researchers find difficult to navigate. In practice, users must explicitly specify resources, node counts, and walltime estimates before submitting jobs to an orchestrator, resulting in iterative trial-and-error that wastes both human effort and compute resources. This thesis proposes an intent-driven orchestration middleware for scientific computing, in which domain scientists express high-level computational goals rather than low-level resource parameters, and the system assumes responsibility for identifying configurations that satisfy those goals efficiently.

This thesis proposal includes the following studies:

- **miniGiraffe: A Pangenome Proxy App (complete):** This work explores a new type of scientific workload on genomics. We performed a performance characterization of this new genomic workload and identified computational bottlenecks. We then developed a proxy application that faithfully mimics the computational behavior of this workload in a lighter, easier-to-port version. We then perform an autotuning experiment on the mapping workflow, in which we found that specializing parameters to inputs and architectures significantly improved the application’s execution.
- **A User-Aware Intent-Driven System (in progress):** Exploring the tuning space for performance improvement on scientific applications can be considerably challenging and hard work. In this research, we will develop a system that allows users to define high-level computational goals when launching scientific jobs. We will explore surrogate models to reproduce the computational behavior of scientific applications, thereby aiding in identifying configurations that achieve the user’s computational intent.
- **A Cluster-Aware Intent-Driven Orchestrator (proposed):** Most of the current resource managers for supercomputing clusters optimize for resource allocation efficiency but ignore higher-level user intents. In this study, we aim to extend the intent-driven orchestration to the cluster level. This system will enable more holistic, policy-aware scheduling decisions that leverage users’ goals with system-wide efficiency.

Chapter 1

Introduction

Over the last few decades, computing has become more accessible and pervasive across society. This growth has brought increasing complexity. As the pace of Moore’s Law has slowed [27], computer and system designers are increasingly turning to alternative strategies to sustain improvements in performance and efficiency, especially for high-performance scientific workflows. Prominent approaches include leveraging hardware heterogeneity through the combination of processors and Graphic Processing Units (GPUs) [46] as well as adopting specialized computing platforms such as field-programmable gate arrays (FPGAs), application-specific integrated circuits (ASICs), and other domain-specific accelerators, such as tensor processing units, neural processing units [26].

With increasing heterogeneity, modern High Performance Computing (HPC) systems have become substantially more complex across multiple hardware and software dimensions. On the hardware side, we observe significant architectural variation across leadership-class machines, with widespread adoption of accelerators, HPC-specific memory technologies such as High Bandwidth Memory (HBM), and high-speed interconnects [35]. An analysis of the Top500¹ architectural trends show that accelerator-based, heterogeneous CPU-GPU nodes have become the dominant design paradigm across leadership-class systems [35], with GPU-accelerated machines representing over 50% of the 500 systems on the latest list [60].

In software, complexity is no longer driven solely by the demands of traditional scientific simulation, such as computational fluid dynamics or finite-element solvers. With the rise and popularity of artificial intelligence (AI), primarily large language models (LLMs) and deep neural networks (DNNs), there is growing interest in integrating traditional scientific simulations with more intelligent solutions, motivated by the need to achieve high-fidelity predictions at scales and speeds that purely numerical methods cannot sustain. AI-coupled simulations for materials science and climate modeling [6, 59], and scientific AI training workloads are some examples of new computationally intensive work-

¹The list which ranks the most powerful supercomputers in the world. <https://top500.org/>.

loads that are reshaping how leadership-class facilities are used, moving beyond the structured, batch-oriented jobs of traditional simulation toward irregular, data-driven, and iteratively adaptive workloads [5]. The increasing use of surrogate models to plan and accelerate scientific workflows can also be cited, as these models replace expensive high-fidelity solvers with computationally cheaper approximations, reducing the overall cost of performing the simulation itself rather than requiring full-fidelity executions at every step [37]. Such rapid growth in AI-enhanced and surrogate-driven workloads is materially influencing HPC system design decisions. As a result, domain-specific accelerators are increasingly being integrated alongside conventional CPU-GPU nodes [26].

This multidomain, heterogeneous scenario intensifies a longstanding challenge in HPC clusters: configuring application simulations to achieve desired performance levels without wasting resources. Deploying and executing scientific applications in modern environments requires navigating a vast configuration space encompassing node count, CPU and GPU selection and allocation, memory binding, and network settings, all of which directly affect execution performance, resource utilization, and the rate at which scientifically meaningful results are produced. Selecting an appropriate configuration is a non-trivial task: suboptimal choices can lead to either underutilization of expensive compute resources or unnecessarily long queue wait times due to overprovisioning [40, 20, 61], not to mention possible wrong outputs from scientific simulations. This challenge is compounded by the growing architectural diversity of leadership-class systems, making optimal configurations highly machine-dependent and rarely transferable across systems [35, 18, 16].

In parallel, the majority of HPC users are domain scientists (e.g., biologists, physicists, climate scientists, among others) whose primary goal is to obtain scientifically faithful results in a reasonable time frame, not to optimize system configurations. Expecting these scientists to also master the intricacies of scheduler policies, memory hierarchies, network topologies, and accelerator configurations is unreasonable: it diverts their attention from the scientific endeavor itself, and the required expertise spans multiple layers of a rapidly evolving system stack that even experienced systems researchers find nontrivial to optimize. Yet, in practice, users must explicitly specify the resources they need before submitting a job to a supercomputer job queue. This requires a substantial number of tests to correctly understand the relationships between workload configuration, resource allocation, execution time, and costs. Users rely on trial-and-error experimentation, iteratively testing different configurations until they achieve acceptable performance. This process incurs substantial overhead in both human effort and queue time, leading to under- or over-resourced allocations at scale.

Such inefficient use of resources is observed in reports from the U.S. Department of Energy (DOE) supercomputers. On Perlmutter, an HPC system at the National Energy Research Scientific Computing Center (NERSC), the report shows that only $\approx 25\%$ of the total CPU jobs launched in November-December of 2022 utilized more than 95% of the CPU power [40]. An analysis of GPU job logs reveals that 15% of jobs have fully idle GPUs, strongly suggesting resource

over-allocation. This is corroborated by a more recent study on the same supercomputer, which finds that many jobs request a full node (4 GPUs) while using only one [11].

Better resource utilization is essential for sustainable HPC systems and advancing science amid surging energy demands. Data centers powering AI and HPC consumed about 415 TWh globally in 2024, representing roughly 1.5% of global electricity consumption. However, as AI grows, this usage is projected to double by 2030. Data center electricity consumption is projected to be around 945 TWh by 2030, slightly higher than Japan’s current electricity consumption [29]. In the United States, clusters account for almost half of the electricity demand. For instance, keeping top exascale supercomputers online 24/7 requires 20-30 MW per facility, equivalent to the annual electricity needs of 15,000-22,000 U.S. homes [62, 57].

Despite the problems that trial-and-error experimentation introduces, an initial exploratory phase is, to some degree, unavoidable when deploying scientific applications on complex systems. It is virtually impossible to determine optimal configurations without empirical observation. Neither the scheduler nor the user can identify performance bottlenecks, resource contention points, or optimization opportunities without conducting preliminary runs. The key limitation is that performance data from these runs is rarely captured systematically or reused. This gap can be addressed by digital twins: users can create virtual replicas of application-system behavior from captured traces, enabling offline what-if configuration testing without real queue submissions. With machine learning integration, this accelerates convergence to optimal allocations, cutting user effort and system waste.

To ensure appropriate use, understanding users’ intent is essential and must be addressed effectively. The goal of this thesis is to establish the foundations of an intent-driven AI orchestrator for heterogeneous, distributed, and resource-aware high-performance computing systems. The central hypothesis is that HPC management and application orchestration can be formulated as a coupled learning and multi-objective optimization problem, whereas high-level, potentially unstructured scientific intents are mapped and translated into system-level decisions under dynamic constraints. This thesis pursues four tightly integrated objectives:

1. **Model Scientific Application Behavior from Execution Traces:** Develop surrogate modeling techniques to capture the relationship between job configuration and performance outcomes using data generated during routine development and testing workflows.
2. **Formalize and Infer Scientific Intent for HPC Systems:** Develop a rigorous semantic and mathematical framework to represent scientific intent (e.g., accuracy, latency, energy budget, and resource constraints). This framework will translate user-provided intent into quantifiable Service Level Objectives (SLOs), bridging the gap between domain scientists and complex computing infrastructures.

3. Develop Scalable Intent-Aware Orchestration: Design AI-driven orchestration methods that map inferred intent to resource allocation decisions, extending scheduling policies into existing HPC resource managers.
4. Autonomous and Interactive Adaptation: Develop closed-loop control mechanisms that leverage streaming system telemetry and human-in-the-loop feedback to dynamically adapt scheduling decisions, ensuring robust satisfaction of SLOs while enabling transparent and trustworthy system behavior.

We posit that future scientific computing systems must operate on intent rather than resources, enabling scalable, adaptive, and resource-aware deployments.

We will address these target objectives throughout this thesis. To start, we provide an overview of the state of the art in the field, the questions being tracked, and the inputs still missing.

1.1 Prior Work

Intent-driven orchestration (IDO) has gained considerable attention in the compute continuum. Compute continuum is the seamless integration of heterogeneous resources spanning Internet of Things (IoT) sensors, edge devices, cloud data centers, and HPC supercomputers [13]. This distributed infrastructure enables dynamic workload orchestration and real-time data processing, but dramatically increases scheduling complexity due to resource heterogeneity, data locality requirements, and dynamic availability. IDO can address such challenges by translating high-level computational objectives into system-level decisions. Works such as [22] and [8] present insightful contributions on allocating serverless function chains across edge-cloud infrastructures, accounting for high-level computational goals rather than specific resource characteristics.

The authors in [22] present an architecture that takes high-level intents (also called service-level objectives (SLOs)) such as latency, cost, and energy, and translates them into parameterized autoscaling and placement decisions for function replicas across edge and cloud clusters. The intent is expressed as objectives, properties, and constraints, and is translated into SLOs, which are then translated into concrete deployments. For the deployments, the solution employs a Deep Q-Network (DQN) with continuous state space and discrete actions, trained offline on traces collected from a random policy and the workload dataset. While this work demonstrates the viability of intent-driven orchestration for optimizing cost and efficiency in serverless deployments, its design is tightly coupled to the elastic, fine-grained resource model of edge and cloud environments.

Similarly, [8] uses intents to focus on reliability and hardware conditions. The authors argue that edge hardware is highly sensitive to humidity, heat, and other environmental conditions, which directly affect performance and hardware reliability. To help address potential availability issues in those scenarios, the

authors propose a reliability, score-based, intent-driven orchestration for edge management. This orchestration system accounts for metrics such as service relocation, telemetry utilization, and server status, making these factors first-class Key Performance Indicators (KPIs) when scheduling services across the network.

[3] proposes IntentContinuum, a framework that uses LLMs to maintain SLO compliance for services deployed across a heterogeneous edge-cloud continuum. The system continuously monitors running services, detects deviations from declared intents, and invokes an LLM to select a mitigation action from a predefined catalog, which is then automatically enforced by the underlying controllers and orchestrators. The predefined catalog includes changing pod placement, scaling replicas, reallocating CPU and memory, or modifying network paths. Satisfying SLOs in a cloud-edge environment is crucial because most tasks occur on the fly and involve decisions that affect people’s lives, e.g., in autonomous driving.

Intent-driven solutions are increasingly adopted in cloud and edge environments, where higher-level abstractions reduce stress on system administrators while ensuring quality of service. As infrastructure becomes more heterogeneous and distributed, the gap between what users need to express and what systems require them to specify grows increasingly untenable. The same mismatch between user expertise and system complexity that drove intent-based approaches in cloud-edge environments is therefore present in HPC, compounded by the additional rigidity of batch scheduling models that offer users little feedback and no elasticity once a job is submitted. Existing AI-driven HPC scheduling work has begun to address resource utilization through predictive models, but stops short of translating high-level user goals into allocation decisions. As a result, both predictive application modeling and flexible scheduling abstractions are prerequisites for an intent-driven HPC orchestrator.

Prior work has begun addressing the modeling aspect of this requirement. [63, 64] present an AI-driven framework that learns from past job executions to model walltime and recommend time- or cost-optimal HPC configurations. By treating applications as black boxes, the HPC Application Runtime Predictor (HARP) uses regression and DNN models to estimate application runtime across different hyperparameters and other inputs. Composed of three components, HARP generates traces from the application using scaled-down and full-scale configurations (generate), and then uses the resulting data table to train the best-fitting model (build). Finally, the system uses the data to predict the optimal execution and resource allocation for the task (predict). This solution closely addresses the application modeling we propose, but focuses solely on runtime. By ignoring other metrics, this framework excludes other potential intentions. Moreover, the best result may not reflect the cluster’s current status, which can delay task execution and possibly the entire pipeline in which the task runs.

In addition to accurate runtime prediction, an intent-driven orchestrator must also rely on a scheduler that can interpret and act on high-level resource descriptions rather than low-level node and partition specifications. Fluxion [49]

introduces a novel graph-based scheduler for HPC systems, aiming to provide more universal and flexible resource management for the new era of supercomputing. One of the main reasons for proposing such a scheduler is to enable greater adoption of the computing continuum on HPC clusters. Traditional schedulers lack a straightforward way to integrate heterogeneous compute hardware with the elasticity required by the computing continuum. By modeling the system as a network of vertices and edges, where vertices represent interchangeable resource pools (e.g., a rack with 100 CPU cores, another with 50 GB memory, etc.) and typed edges denote relationships such as containment or conduits across subsystems, it is possible to perform dynamic updates to the available hardware. This dynamic approach can also better exploit hardware resources when the user describes specific resource needs rather than partitions or nodes. Fluxion therefore provides a natural foundation for an intent-aware cluster, enabling higher-level user definitions when launching a job. However, it stops short of directly interpreting semantic user goals, leaving the translation from intent to concrete resource specification an open challenge.

1.2 Contributions and Proposal

This dissertation explores how intent-driven orchestration can transform HPC scheduling from resource-centric to outcome-centric, leveraging application behavior learned from users’ routine testing workflows. The central goal is to simultaneously improve usability for domain scientists, application performance, cluster utilization, and energy efficiency by shifting from manual, resource-aware configuration to automated, intent-driven recommendations. This approach exploits the rich performance data already generated during the standard HPC development cycle (e.g., small-scale testing, parameter sweeps, and load-balance exploration) to construct application-specific predictive models that guide scheduling decisions without requiring users to master scheduler-specific parameters.

So far, my main contribution has been competing and publishing the following paper:

- Jessica I. Dagostini, Joseph B. Manzano, Tyler Sorensen, and Scott Beamer. miniGiraffe: A Pangenomic Mapping Proxy App. In 2025 IEEE International Symposium on Workload Characterization.

In this work, we present miniGiraffe, a proxy application based on the VG Giraffe pangenome mapper, to facilitate testing of such workloads across various systems. We have applied a workload characterization and identified the critical regions from Giraffe. With that, we decoupled it and developed a proxy application that faithfully mimics the parent’s behavior in a lighter, easier-to-port version. The simplified design of miniGiraffe enabled us to perform an autotuning experiment on the mapping workflow, in which we found that specializing parameters to inputs and architectures

significantly improved the application’s execution. We observed how challenging it can be to find such parameters, and that applying them can improve application performance and hardware efficiency.

The ongoing and planned research this proposal includes:

- A User-Aware Intent-Driven System. Potential submissions for ACM/IEEE SC’28.

From the results of the exploratory work on miniGiraffe autotuning and load-balancing efficiency in AMReX applications, we understand the impacts and challenges of identifying configurations that achieve different performance outcomes. Focusing on addressing such performances in accordance with users’ intents, we propose a system to help identify the best configuration for running a given application based on high-level goals. Rather than declaring specific parameter values or node counts, the core of this system is to allow the user to define service-level objectives and let the system determine the specific low-level configuration needed to achieve them. We demonstrate the potential of this idea by conducting a feasibility study on WarpX, drawing on insights from the previous load-balance exploration, and integrating statistical and machine-learning approaches to estimate runtime and balancing efficiency, thereby enabling the definition of high-level intents for these aspects.

- A Cluster-Aware Intent-Drive Orchestrator. Potential submissions for ACM International Symposium on High-Performance Parallel and Distributed Computing 2028.

In this proposed research, we aim to extend the user-aware, intent-driven system to the cluster, bringing intent awareness directly into HPC resource managers. Rather than treating intent middleware as a front-end that produces conventional job scripts, we propose exposing intent metadata to the orchestrator itself, enabling it to reason jointly about application service-level objectives, user goals, and cluster policies. By bringing these capabilities across the entire cluster, this approach seeks to both improve the realization of user intent and enhance overall resource utilization, effectively extending existing job scheduling frameworks with a more comprehensive, intent-aware design.

Chapter 2

Background

In this chapter, we provide the necessary background in high-performance and scientific computing, focusing primarily on resource management and allocation, scheduling, performance portability, and application profiling. All these topics serve as the baseline for the proposed work. We conclude this chapter by providing the necessary background on intent-based systems and its terminology.

2.1 High Performance and Scientific Computing

Scientific challenges such as running climate simulations, DNA sequencing analysis, and large-scale physics simulations are emblematic of the complex problems that High Performance Computing (HPC) systems are designed to address. HPC systems combine high-speed processors, high-performance networks, and large memory capacity to enable workloads that would be computationally intractable on conventional systems.

These systems, however, have not remained static. Early supercomputers were largely homogeneous: thousands of near-identical nodes sharing the same CPU architecture and memory configurations, connected by a high-speed fabric. As Moore’s Law approached its physical limits, sustaining performance gains required a fundamental architectural shift toward heterogeneous hardware [9]. Today, the Top500 list reflects this reality, with leading systems combining CPUs and GPUs across distinct intra- and inter-node interconnects [35]. The rise of AI-at-scale has deepened this transformation: as demand for large-scale machine learning has grown, clusters have incorporated increasingly specialized and custom hardware configurations tailored to specific workload classes [51]. Despite often running on the same GPU hardware, HPC simulations demand high-precision floating-point arithmetic, while AI workloads exploit reduced-precision formats, such as FP16, BF16, and quantized integer types, that scientific codes cannot adopt without sacrificing correctness. This divergence has led hardware vendors to dedicate an increasing amount of silicon area to lower-precision compute units [33], producing systems whose hardware capabilities are increasingly

mismatched to any single workload type. This architectural fragmentation compounds an already difficult resource management problem: clusters must now serve workloads with fundamentally different hardware affinities, precision requirements, and scheduling models within a shared infrastructure.

Given their size and cost, high-performance clusters are shared among hundreds or thousands of users. In practice, it is uncommon for real-world scientific applications to demand the entire cluster simultaneously. However, many traditional HPC and scientific applications are highly sensitive to resource contention, which can significantly degrade performance when workloads compete for the same set of resources [20]. As a result, nodes must often be allocated exclusively to a single job for the duration of its execution to prevent interference [20]. Moreover, the growing integration of AI-driven, data-intensive workloads alongside traditional batch jobs introduces fundamentally different scheduling requirements within the same facility, including tenant isolation, privileged provisioning, and mixed orchestration across batch schedulers and cloud-native service management [58]. This tension between shared infrastructure, exclusive access requirements, and the co-existence of heterogeneous workload types is precisely why resource allocation and job scheduling have been fundamental to HPC clusters from the very beginning. It also explains why they remain a significant challenge, particularly in heterogeneous and AI-augmented HPC environments.

2.1.1 Resource Allocation and Scheduling

Typically, users of an HPC cluster submit their jobs to a queue, managed by a job scheduler [71]. Such jobs are defined in batch scripts that contain all the necessary information to run them, including the number of computing nodes/resources required, and a runtime estimate. Depending on the target system, the script may also specify a resource queue (for example, a dedicated GPU queue) and the project account to which the job will be charged. Many leadership-class facilities implement a fair-sharing policy, allocating a finite number of node-hours per project [36], thereby making account specification a necessary component of job submission. While the specific factors vary across HPC facilities, this general formulation is representative of node-hour-based accounting commonly used at leadership-class computing centers:

$$\text{walltime in hours} \times \text{number of nodes} \times \text{QOS factor} \times \text{charge factor} \quad (2.1)$$

where QOS stands for quality of service factor.

Despite a few job scheduling solutions on the market, only two have gained broader adoption. The most popular HPC resource manager and scheduler is the Simple Linux Utility for Resource Management (Slurm) [71]. Developed by the Lawrence Livermore National Laboratory, this open-source solution has consistently ranked as the most widely used scheduler in the Top500 list [30]. Slurm supports exclusive and shared node allocation, enabling users to schedule and monitor jobs, and coordinates resource contention through managed job queues. Another popular scheduler is the Portable Batch System (PBS) [25]. Directly

descended from the Networking Queueing System (NQS), developed by NASA, this orchestrator handles job lifecycle management, resource management, job scheduling, and execution status monitoring. Both schedulers adopt the same declarative model described above, requiring users to explicitly define resources and estimated runtime prior to submission.

These traditional scheduling models are typically node-centric, using bitmap, linked-list, or simple static graph implementations to manage allocation control [49, 71, 25]. At their core, these schedulers face an NP-hard combinatorial optimization problem: assigning jobs to machines to minimize the overall makespan (i.e., the total time to complete all submitted work). As an exact solution to this problem is computationally infeasible, practical schedulers rely on heuristics to approximate good allocations. One of the most widely adopted is backfilling, a technique that allows smaller or shorter jobs to be scheduled ahead of larger, waiting jobs, provided they do not delay the expected start time of those higher-priority jobs [41]. Backfilling comes in two main variants: conservative backfilling [61], which preserves the reserved start times of all queued jobs, and Extensible Argonne Scheduling sYstem (EASY) backfilling [55], which only guarantees the start time of the first job in the queue and is more aggressive in filling gaps. While backfilling significantly improves cluster utilization in practice, it also reinforces the dependency on accurate walltime estimates.

This dependency becomes increasingly problematic as the underlying hardware grows more heterogeneous. The above-mentioned solutions were based on a far less complex HPC system than today, where every compute node had similar characteristics and job elasticity was not a key design consideration. For instance, if a scientific workflow requires GPU acceleration only during a specific execution phase, traditional schedulers do not support dynamic reallocation of those resources once the GPU-intensive phase completes. Contemporary supercomputing systems are no longer homogeneous collections of CPU cores; they integrate diverse hardware accelerators, heterogeneous memory hierarchies, high-speed interconnects, and increasingly, AI-optimized processing units [5]. As specialization continues, broad HPC adoption requires managing a wide range of heterogeneous components, each optimized for a specific domain, while allowing workloads to navigate between them. This architectural diversity compounds the already difficult task of resource estimation, making optimal utilization increasingly difficult.

Beyond algorithmic difficulties, a critical yet often overlooked dimension of this problem is the nature of the primary HPC user base. The vast majority of HPC users are domain scientists in fields such as climate modeling, molecular dynamics, astrophysics, or computational biology, whose primary objective is to obtain scientifically valid results rather than to optimize resource utilization [56]. This represents a fundamental mismatch between the design assumptions of current scheduling systems and the expertise of their primary user base, as it requires users to have specialized knowledge to determine which computing resources they need and for exactly how long. In practice, users frequently engage in an iterative submission cycle, repeatedly adjusting resource requests based on prior execution failures or suboptimal outcomes.

Given this reliance on manual resource estimation, users frequently over- or underestimate the total resources and walltime required for a given job. This overestimation leads to suboptimal resource use in the cluster. HPC clusters have been shown to operate at 35–50% CPU utilization and below 50% memory utilization on average [40]. When evaluating the accuracy of job runtime estimation, studies report that this accuracy is typically less than 60% [20, 66, 68]. Those studies also indicate that most jobs use only an average of 29% of the requested runtime. This overestimation is often deliberate: faced with the risk of having a job killed mid-execution for exceeding its time limit, users commonly default to requesting the maximum allowed walltime as a safety measure [15]. Addressing this gap between the complexity of modern HPC systems and the scheduling abstractions available to domain scientists is a central motivation for an intent-driven orchestrator.

2.1.2 Performance Portability and Application Profiling

Securing compute time is only the first step toward productive use of HPC systems; the application itself must be carefully adapted to the target system’s hardware architecture. In the early homogeneous era of HPC, a code written for one cluster could reasonably be expected to perform adequately on another. Now, given the heterogeneous landscape described earlier, an application optimized for one system may perform poorly or even fail entirely on another. Pennycook et al. formalize this as performance portability: an application is considered performance-portable if it achieves similar performance and correct behavior across a range of target architectures [50, 18]. This portability has become a central concern in computational science.

For this reason, conclusions and optimizations made not only at the application level, but also regarding resource allocation to run the application in a cluster, need to be applied to each specific supercomputing facility. Given their complexity, such optimizations cannot be obtained solely through intuition. This requires systematic collection and analysis of runtime performance data. Performance tracing tools instrument an application’s execution to record time-stamped events, such as function calls, memory transfers, synchronization points, and communication operations, producing traces that reveal the application’s dynamic behavior at runtime [24]. These traces are essential for identifying load imbalances, communication bottlenecks, suboptimal memory access patterns, and underutilized hardware units. Without this information, tuning efforts are effectively blind, and the application adaptation process reverts to the same trial-and-error dynamic that characterizes job scheduling.

Collecting performance traces at scale introduces its own set of challenges. One is to ensure that the instrumentation does not perturb the application’s behavior. The act of measurement itself can alter resource consumption and timing, obscuring the very phenomena under study [24]. Another challenge is matching instrumentation tools to an application. The diversity of programming models (MPI, OpenMP, CUDA, HIP) and hardware architectures makes it difficult to deploy a single instrumentation tool uniformly across different

facilities and codebases, often requiring application-specific adaptations.

Many scientific frameworks have addressed these instrumentation challenges by developing application-specific profiling tools tailored to their execution models and hardware targets. In the block-structured adaptive mesh refinement AMReX framework, for instance, the TinyProfiler provides lightweight, hierarchical timing instrumentation that records per-routine wall-clock time across MPI ranks with minimal overhead [72]. While such embedded profilers offer convenience and low perturbation, they are generally limited in scope, capturing application-level timing but generally not exposing lower-level hardware performance counters, memory bandwidth utilization, or network communication patterns. A complete understanding of application behavior, therefore, typically requires combining framework-level profiling with external tools such as Scalasca [24], Score-P, or NVIDIA Nsight, each targeting a different layer of the execution stack. However, the performance insights derived from this multi-layered profiling process are system-specific: optimizations validated on one supercomputing facility do not necessarily transfer to another, and the resource configurations that yield efficient execution are rarely known a priori by the domain scientist submitting the job.

In summary, the challenges discussed in this section operate at multiple levels of the HPC stack. At the hardware level, increasing heterogeneity expands the configuration space that both schedulers and applications must navigate. On resource usage, the NP-hard nature of job scheduling forces schedulers to rely on heuristics that depend on walltime estimates that users are generally unable to provide accurately. At the application level, performance portability requires system-specific profiling expertise that most domain scientists do not possess. Thus, a consistent pattern emerges: domain scientists are expected to make precise, low-level infrastructure decisions that lie outside their primary area of expertise. This complexity motivates an alternative paradigm, in which users express what they need from the system to achieve their scientific goal, and the system assumes responsibility for making the low-level configuration decisions. The following section examines intent-based systems as a conceptual and technical foundation for such an approach.

2.2 Intent-based systems

The concept of the intent-based management paradigm originated in the networking domain as Intent-Based Networking (IBN) [39]. IBN was developed to let operators specify desired outcomes, such as secure access for a specific department, rather than manually configuring IP routes, VLAN assignments, or Access Control List rules. The system is then responsible for translating the declared intent into the necessary underlying configurations, continuously monitoring the infrastructure at runtime, and automatically adapting to maintain the desired state.

Building on this networking foundation, Intent-Driven Orchestration (IDO) extends the IBN paradigm to computing infrastructure, enabling intent decla-

rations to be expressed directly within workload specifications [45]. The work introducing IDO also presents a planning component embedded in the Kubernetes control plane that continuously compares the current state of a workload instance against the desired state defined by its objectives. Most cloud-native environments operate on a declarative resource model, in which application owners specify the exact quantities of hardware resources (CPU, GPU, memory, etc.) their applications require before deployment. An intent-driven system shifts this paradigm by allowing users to submit requests in terms of Service-Level Objectives (SLOs), such as low latency when requesting a service or high service availability, which are tied to key performance indicators (KPIs) [45]. The orchestrator will then work to meet users’ SLOs by tracking KPIs from a knowledge base that comprises observed metrics and modeled predictions, without requiring users to understand the underlying infrastructure details. Notably, existing IDO implementations target cloud-native environments and have not been adapted to the batch-oriented model used by HPC schedulers.

Both intent-based systems paradigms share a common four-phase operational lifecycle: intent profiling, intent translation, intent resolution, and intent assurance [39]. Intent profiling captures the user’s high-level objective and structures it into a machine-interpretable form. Intent translation converts that structure into concrete configurations or resource actions. Resolution detects and arbitrates conflicts between simultaneously active intents (when two workloads compete for the same constrained resource with contradictory latency targets). Finally, intent assurance continuously monitors the live infrastructure and re-triggers the translation and resolution pipeline whenever a violation is observed, implementing a closed-loop feedback mechanism [12].

In the specific case of the IDO for Kubernetes, each intent also has a planner that determines all possible actions from the current state to the desired state, building a state graph. A shortest-path algorithm is then applied to the graph to determine the potential set of actions [45]. A critical enabler of the state graph planner is the ability to quantify the distance between the current and desired application states. De Souza et al. [38] develop a family of distance functions for this purpose, ranging from simple Euclidean measures across KPI objectives to weighted and bounded formulations that account for objective priority and directionality. These functions serve as heuristics for A* search to reduce the number of candidate states and utility cost inputs, guiding the planner toward the action sequence that most closely satisfies the declared intent.

While the original IDO work targets single-cluster Kubernetes environments, the paradigm has been extended to other configurations and to new objectives. The compute continuum (the heterogeneous spectrum of IoT devices, edge nodes, and large-scale cloud and HPC centers that modern applications increasingly span) presents an especially compelling context for intent-based approaches, since cloud computing alone does not suffice for many real-time and privacy-sensitive applications, prompting the integration of edge and cloud resources for seamless operation [13]. By allowing users to define their desired outcome rather than prescribing low-level infrastructure configurations, intent-based orchestration simplifies management, enhances automation, and bolsters

overall system agility in these environments [4].

However, extending intent-based orchestration to the compute continuum introduces substantial new challenges. The continuum features diverse capabilities, locations, programming models, and hardware constraints that considers computing power, storage, networking, and specialized accelerators, with resources spanning multiple administrative domains governed by distinct owners and policies [22]. Non-functional requirements such as availability, latency, and cost must be satisfied simultaneously across this wider and more heterogeneous platform, and the resulting growth of the intent planner’s state space, both in size and dynamism, makes naive extensions of single-cluster approaches computationally intractable [3]. Proposed solutions address these challenges by adopting a hierarchical, multi-domain model that partitions the continuum into logical domains (compute, network, and storage), each managed by a local tool. A coordinator layer sits above these domain managers, maintaining a global view of the intent and assigning concrete per-domain target objectives that, when satisfied, fulfill the application’s high-level goal. The coordinator additionally handles cross-domain conflict detection, dynamically adjusting targets as infrastructure state evolves and learning over time to produce globally optimal assignments [4]. This hierarchical decomposition is a natural structural parallel to the multi-level resource management challenge in heterogeneous HPC environments, where facility-aware scheduling decisions must be reconciled with application-aware performance objectives.

A further extension of the intent-driven scope incorporates declarable intents that encompass sustainability as a first-class orchestration objective [44]. This extension is a natural consequence of the IBN/IDO paradigm itself: if the orchestration stack is already capable of autonomously pursuing arbitrary measurable objectives, then environmental metrics such as carbon efficiency, energy consumption, and greenhouse gas emission rates can be incorporated as additional intent targets without requiring fundamental architectural changes [45]. These metrics are formalized as Key Value Indicators (KVIs), providing a declarative vocabulary for expressing sustainability constraints within the same intent specification structures used for QoS objectives. In the context of HPC, where large-scale facilities consume substantial energy and operate under increasing pressure to demonstrate environmental accountability, the ability to express energy constraints as declarable intents, alongside performance objectives, represents a meaningful extension of the scheduling paradigm addressed in this thesis.

Despite the exploration of intent-based approaches for cloud and continuum environments, no existing work systematically applies the IDO paradigm to the batch-scheduling model of HPC clusters. We further argue that IDO can be a natural addition to enable leadership-class facilities to support heterogeneous workload types. The scheduling abstractions available in HPC are precisely the low-level imperative specifications that intent-based systems are designed to abstract. The work proposed in this thesis aims to address this gap by adapting the intent translation, resolution, and assurance lifecycle to the constraints and workload characteristics of large-scale scientific computing.

Chapter 3

Completed Research

This chapter presents and discusses the findings of two studies conducted during this PhD program. The first focuses on miniGiraffe, a proxy application for pangenome mapping. In this work, we characterize the performance behavior of a pangenome mapping application and leverage those insights to develop a representative proxy app. Using miniGiraffe, we then investigate how different tuning parameters affect performance across a range of test cases and computational environments. The second study, conducted during a summer internship, explores load-balancing techniques for applications built on the AMReX framework. In this work, I contributed to the development of a toolbox that simulates the behavior of various load-balancing algorithms under a given weight distribution on an AMR grid. Both works inform the proposed research presented in the next chapter.

3.1 miniGiraffe: A Pangenome Proxy App

Scientific applications are generally hard to profile due to the complexity of their codebases. They are normally coded focusing on an architecture-specific approach and normally depend on a vast quantity of libraries with fixed versions, which can make its portability across environments and clusters difficult. To manage this complexity, *proxy applications* (often referred to as *proxies* or *miniapps*) are commonly employed. These are simplified versions of full applications that retain key characteristics such as computational kernels, memory behavior, or communication patterns; and as such, these proxies can be used to understand system behavior and, in turn, develop optimization strategies [1]. For example, proxies have been used to improve the energy efficiency of large physics simulations [70] and to evaluate different programming models in shock hydrodynamics applications [31].

Emerging fields present new computing challenges, as with pangenomes. Pangenomes aim to represent the entire set of genes within a population by building a genome reference from cross-references between hundreds (and, in

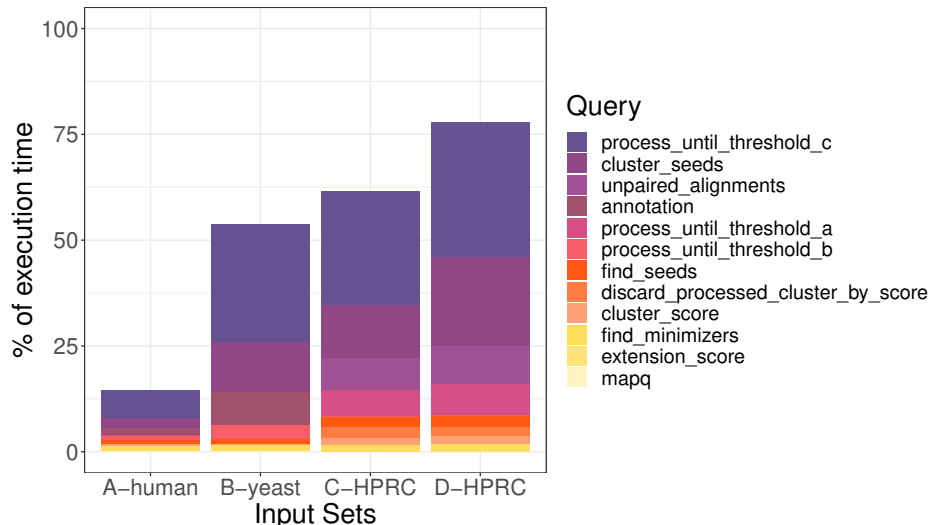


Figure 3.1: Percentage of runtime for each instrumented region for all input sets. Times are aggregated per thread, and averaged across threads for each input.

the future, thousands) of different DNA sequences. Pangenomes can be used at many scales, whether it be a population, species, or even a metagenome [19, 47]. In this work, we consider the complex pangenome mapping application *VG Giraffe* [54]. This application uses *variation graphs*[23], a bioinformatics data structure to represent linear genome data and their respective variations. Giraffe is part of the larger toolkit *VG*, which contains several other applications that utilize variation graphs, and its mappings perform many traversals over the graph. These tools all have complex inter-dependencies, making focused analysis and optimization difficult. We first tackle this challenge to understand what mostly impacts Giraffe performance.

Characterizing Giraffe We performed a detailed performance analysis of VG Giraffe to understand its bottlenecks. We combined standard profiling tools with custom instrumentation and ran VG Giraffe with four different input sets that covered all possible workflows within the application. Overall, we observe in Figure 3.1 that the *process_until_threshold_c* region was the most time-consuming in all executions, varying from 7% to 32% on input sets A and D, respectively. Moreover, we observe a significant increase in execution time for the *threshold_c* function when processing a larger number of reads (input sets B-yeast and D-HPRC). Additionally, we note that VG’s preprocessing causes significant overhead when running small read datasets (input set A-human). If we disregard the IO time, the *threshold_c* function represents 46.4% of total computation time for the input set A-human and 52% on B-yeast.

Table 3.1: Configuration parameters for fastest results, including batch size (BS columns), CachedGBWT capacity (CC columns), and scheduler (OpenMP unless marked with * for work-stealing).

Input Set	local-intel		local-amd		chi-arm		chi-intel	
	BS	CC	BS	CC	BS	CC	BS	CC
A-human	256	4096	512	2048	256	4096	128	2048
B-yeast	128	512*	128	512	256	1024*	128	1024*
C-HPRC	256	4096	128	4096	512	256*	128	4096
D-HPRC	128	2048	1024	4096	1024	2048*	512	1024

Building miniGiraffe Based on these findings and on our experience with playing with the full scientific application, we developed *miniGiraffe*, a proxy application for Giraffe. *miniGiraffe* encapsulates these critical functions, with I/O provided by capturing I/O to those functions in Giraffe. The main goal of *miniGiraffe* is to serve as a portable, lightweight testing tool for pangenome-mapping workflows. With this tool, we were able to further investigate the impacts of different parameters and their relation to the hardware on performance.

Playing with tuning One of the main contributions of this work is to demonstrate how tuning parameters affect the performance of this pangenome-mapper workload across different hardware platforms. We expose three parameters: the scheduling technique, the batch size, and the initial CachedGBWT capacity. GBWT is the acronym Graph Burrows-Wheeler Transform, an efficient index implementation that enables efficient storage and queries over the pangenome [53]. These modifications exposed and enabled experimentation with parameters that are not easily adjustable in the original Giraffe tool (e.g., a C++ thread-based work-stealing scheduler, batch size (which defaults to 512), and CachedGBWT initial capacity, which is originally hard-coded to 256).

We conducted an auto-tuning experiment across four different servers to understand the impact of these parameters on this workload and potential benefits. More information on server characteristics is available in Appendix A. Two were local servers (**local-intel** and **local-amd**), and two of them (**chi-arm** and **chi-intel**) are from the Chameleon Cloud testbed [34]. To run all four input sets on all machines across all parameter combinations, we subsampled the input sets. For each input set, we used only the first 10% of reads. We exhaustively auto-tune (full cross-product) across scheduler, batch size, and initial CachedGBWT capacity. For the batch size, we considered powers of 2 from 128 to 2048. For each platform, we use the maximum number of available thread contexts (including hyperthreads): 96, 128, 64, and 160 on local-intel, local-amd, chi-arm, and chi-intel, respectively. For this analysis, we consider the *makespan*, i.e., the end-to-end execution time. For each system and input, we identify the best-performing set of parameters.

By tuning the parameters, we achieved significant speedups across all input sets on almost all systems. Interestingly, most of the best performers do not

share the same configuration, nor use the default values (OpenMP, 512 batch size, 256 initial CachedGBWT capacity). Despite our focus on lightweight performance, our work-stealing scheduler is the fastest on only 5 of 16 scenarios, with 3 of them on the chi-arm system. The most significant overall speedup from all of our tuning was $3.32\times$, and it was achieved on input set A on the chi-arm system. The smallest overall speedup was $1.01\times$, and it occurred on input set B on the local-amd system. In fact, input set B-yeast only benefits substantially from tuning on the chi-intel machine. Only input set C-HPRC presented a smaller variability in speedup between systems. The geometric mean speedup for each input set is respectively $1.36\times$, $1.07\times$, $1.10\times$, and $1.11\times$.

To understand the nature of the parameter space, we plot the makespan

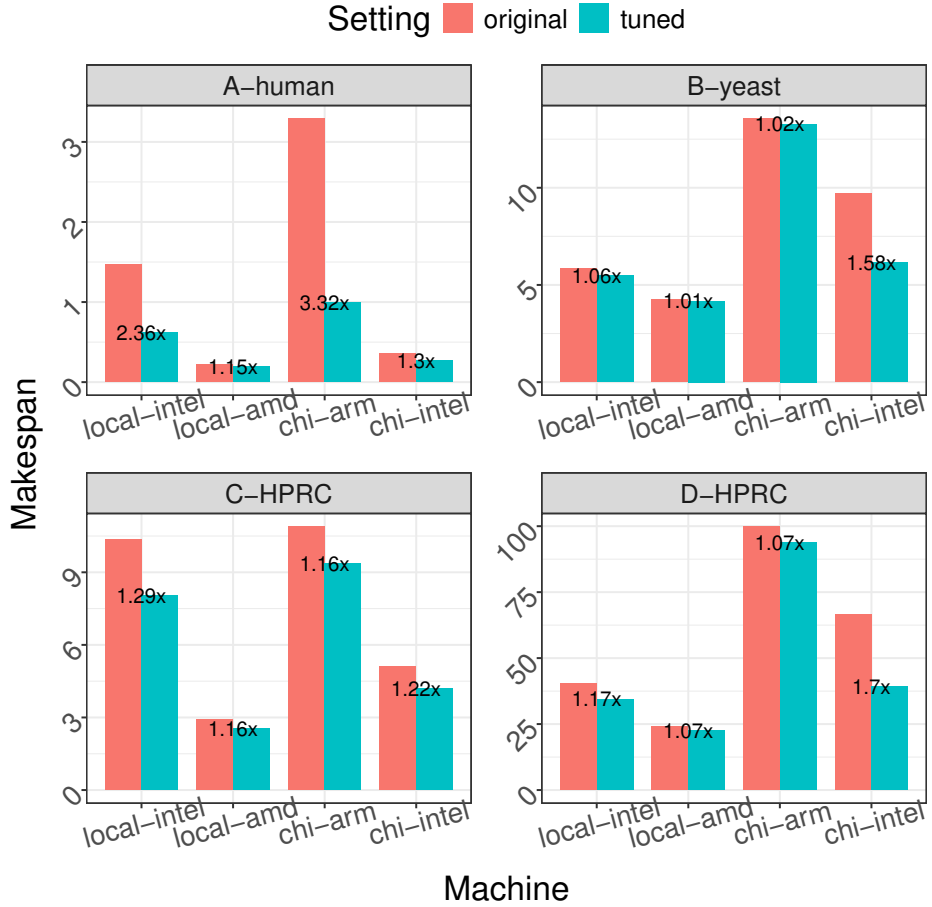


Figure 3.2: Makespan (seconds) comparison of the best tuning performers against the default parameters for each input set over each machine. Executions use all available threads on each machine, with counts of 96, 128, 64, and 160 for local-intel, local-amd, chi-arm, and chi-intel, respectively.

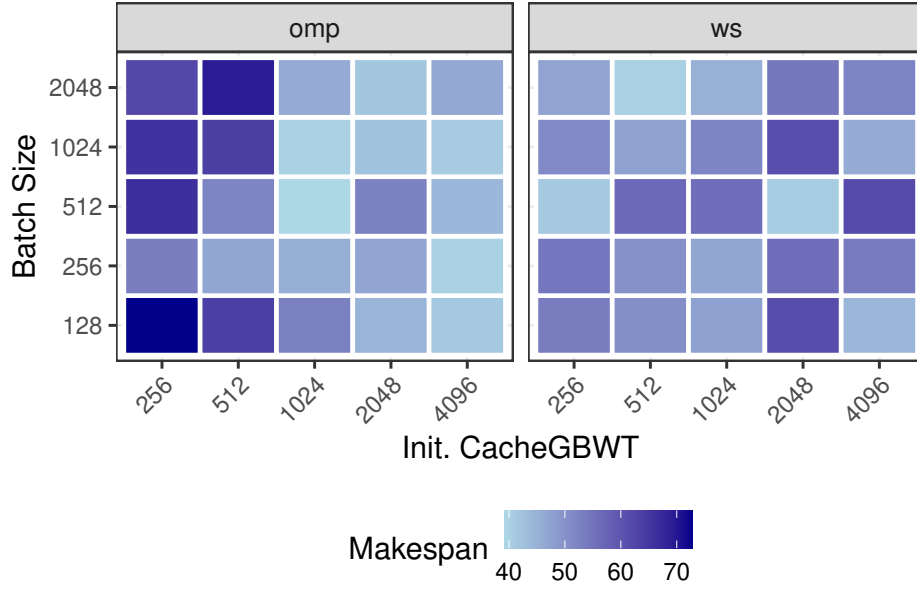


Figure 3.3: Heat map with makespan distribution of all different combinations of tuning parameters over input set D-HPRC at chi-intel machine. Makespan is set in seconds. The left facet shows results for the OpenMP approach (omp), while the right facet shows results for the work-stealing (ws) approach. Lighter blue indicates faster runtime, while darker blue squares indicate slower runtime.

across all parameter combinations for input set D-HPRC on the chi-intel system (Figure 3.3). We selected this system since it had the largest variance in parallel scalability and this input since it is the largest. We observe a significant difference between the best and worst performers on this system. Most of these combinations significantly impact performance, demonstrating the importance of parameter tuning. By choosing the best tuning combination, it is possible to avoid a $1.76\times$ slowdown for this input set on this system. More interestingly, the default parameters produce one of the slowest executions.

3.1.1 Summary

In this work, we investigate the performance of an emerging scientific workflow, VG Giraffe, identify challenges in its analysis, and examine its key component kernels. We then developed miniGiraffe, a proxy application that faithfully mimics Giraffe’s critical functions in an easier-to-play manner. We then conducted an exhaustive analysis and auto-tuning on miniGiraffe, identifying significant differences in parameter tuning across different architectures. From this work, we demonstrate how challenging it is to achieve optimal performance in scientific applications, particularly given the heterogeneity of current hardware.

3.2 A Toolbox for Load Balancing Simulation for AMReX Applications

Efficiently balancing workloads is crucial for scaling scientific applications. While achieving maximum hardware utilization is key, testing new load-balancing strategies can be complex due to challenges in collecting and analyzing performance data. In this work, we aim to address this workflow challenge by presenting a toolbox to streamline the collection and analysis of load-balancing data from WarpX, an advanced, fully kinetic particle-in-cell (PIC) code [21]. WarpX is based on AMReX, a block-structured adaptive mesh refinement (AMR) software framework primarily used for solving partial differential equations [73]. As part of its functionality, AMReX provides load-balancing algorithms to efficiently distribute AMR “boxes” across threads or GPUs based on their computational costs, improving software scalability. These boxes are refinements of the grid produced by the code, breaking the entire grid into smaller boxes with minimum and maximum grid sizes defined by the execution input.

This toolbox leverages one of WarpX’s “Reduced Diagnostics” to collect load-balancing costs. These costs can be either embedded timers or a “heuristic” (a weighted counting of meshes and particles in each box). After defining the input parameters, the toolbox runs a simulation, records costs throughout execution, and writes the data to a CSV file. With the parsed balancing costs, users can run different load-balancing algorithms through our toolkit to better understand their applicability and make informed decisions about optimal parallelization strategies. Figure 3.4 shows the toolbox’s workflow.

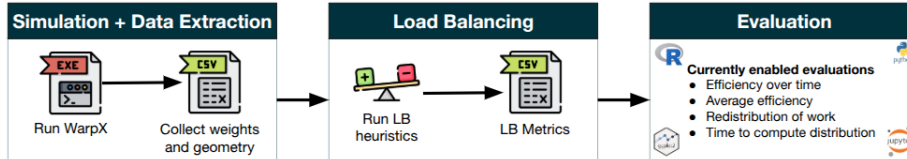


Figure 3.4: Workflow applied at our LB toolbox.

Currently, this toolbox supports strategies in two main categories: partitioning and communication-approximation algorithms. Two partitioning algorithms distribute AMR boxes to workers based solely on each box’s cost:

- Knapsack: a greedy algorithm that first orders the boxes by weight, and then assigns the next heaviest box to the lightest rank. AMReX/WarpX supports this strategy [52].
- Karmarkar-Karp [32] (a.k.a. least difference): repeatedly takes the two largest numbers/tuples from a set, replaces them with their difference, and sorts the list. The tuples are formed by repeatedly assigning the largest number to the subset with the smallest sum to minimize the total difference.

Communication-approximation algorithms distribute AMR boxes to workers based on their location on the grid, grouping proximate boxes to reduce communication costs. Two such solutions are implemented:

- Space Filling Curve (SFC): Also supported by AMReX/WarpX [52] as the Z-order SFC, this strategy orders the boxes by their proximity in the grid and distributes them by a proportional allowable weight (average). We extend this implementation by adding a Hilbert SFC strategy.
- Painter’s Partition: using a similar ordering approach to SFC, this method employs a binary search to find the most efficient partitioning of work. The algorithm searches for a minimum ”target weight” (or maximum time). If a partition is possible within the target weight, it reduces the target to find an even faster solution; otherwise, it increases the target [48].

The toolbox also includes strategies from a previous study on AMReX load balancing, combining communication-approximation with partitioning [48]. There are two hybrid implementations: pairing the Z-Order SFC algorithm with Knapsack, and pairing the Painter’s solution with Knapsack. Both use the SFC algorithm for inter-node communication and Knapsack to efficiently distribute the workload locally on each node. We also implement a Painter’s solution using the Hilbert SFC strategy.

To demonstrate the applicability of our toolbox, a ”Laser Ion Acceleration with a Planar Target” simulation [10, 43, 69, 67]. This benchmark models laser-ion acceleration with planar solid-density targets, simulating the interaction between ultra-high-intensity laser pulses and matter, which is a physically demanding scenario that requires fine spatial and temporal resolution to capture plasma dynamics accurately. This is a commonly used test for performance-oriented studies, and its documentation explicitly highlights the use of dynamic load balancing. For this demonstration, two different layouts were used, both with a maximum box size of 512. The first domain (2688 x 3712, 48 boxes) was tested on 4 and 8 GPUs, while the second (7488 x 14720, 435 boxes) was run on 24, 48, and 96 GPUs. Both layouts evolved over 5000 steps, with load-balancing metrics collected every 100 steps. Figure 3.5 demonstrates the load balancing efficiency over the simulation’s execution for each of the six solutions. This is an initial example of a comprehensive overview of the balancing algorithms that our toolbox allows.

From these results, it is clear that some algorithms, such as Knapsack and Karmarkar-Karp, exhibit more peak regions than others. The pure SFC solution also shows high peak values. These peaks represent redistributions of work to improve efficiency by at least 10%. Beyond this, the toolbox readily enables a range of further analyses: users can study how varying the minimum efficiency threshold affects redistribution frequency across algorithms, and can correlate redistribution events with the resulting efficiency to identify opportunities for improving application performance.

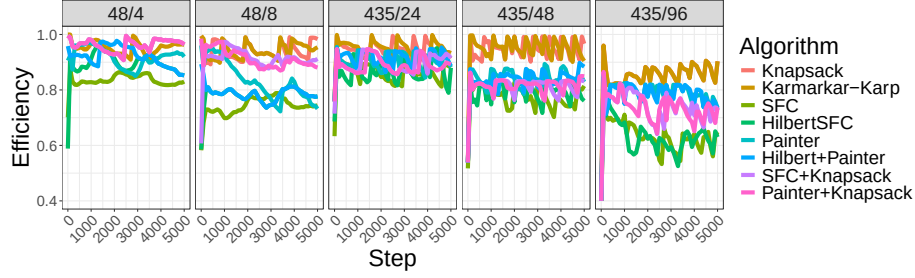


Figure 3.5: Load balancing efficiency over the execution of the simulation. Each step represents a phase in the original simulation during which load-balancing costs were collected. Each panel corresponds to a distinct ratio of domain boxes to GPU ranks (48 boxes/4 GPUs through 435 boxes/96 GPUs), with efficiency measured over successive time steps.

3.2.1 Summary

In this work, we presented a load-balancing toolbox designed to lower the barrier to exploring and comparing load-balancing strategies in AMReX-based applications. Users can focus directly on algorithmic analysis, investigating how different solutions affect the frequency of redistribution, load-balancing efficiency, and parallel execution. The toolbox supports reproducible and extensible analyses, facilitating a more systematic study of load balancing in AMR-based scientific simulations. Additionally, load-balance efficiency is a natural candidate for an intent target, and the toolbox developed here can provide the measurement and simulation layer that makes this intent measurable.

Chapter 4

Proposed Research

This chapter presents two sections of proposed future work that, while closely related, differ in their focus:

1. **User-Aware Intent-Driven Middleware** In this proposed research, we aim to model the application’s behavior to understand how different tuning parameters and metrics affect its performance. We plan to leverage the execution traces generated during development and testing phases to construct surrogate models of application behavior. Drawing on these insights, we will design intent-driven middleware focused on identifying the most suitable configurations to achieve high-level intents. The proposed middleware will allow users to specify high-level Service Level Objectives rather than low-level scheduler parameters.
2. **Cluster-Aware Intent-Driven System** Building on the user-aware intent-driven system, we aim to bring intent awareness directly into the scheduling layer by extending existing HPC resource managers. In this work, we plan to combine user- and cluster-aware intents to drive better resource utilization and computational efficiency. Rather than treating the intent middleware as a front-end that produces conventional job scripts, this work plans to expose intent metadata to the orchestrator itself, enabling it to reason jointly about application SLOs and facility policies.

4.1 User-Aware Intent-Based System

Before deploying an application at scale, domain scientists typically need to conduct exploratory runs to understand its performance characteristics, assess the impact of tuning parameters, and determine whether it can deliver the desired scientific output. When developing new solutions, development teams run multiple tests to verify the application’s behavior. The tuning experiments on miniGiraffe and the load-balancing exploration on WarpX demonstrate that

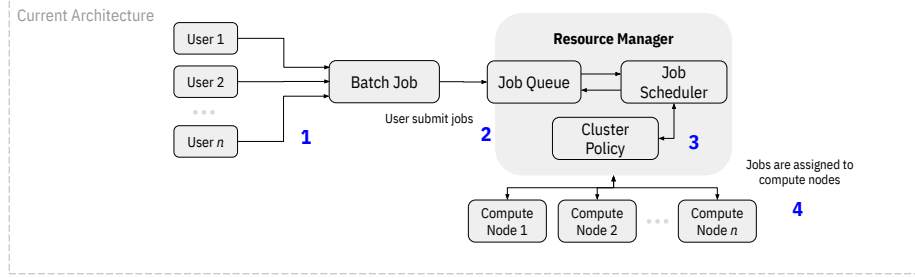
application- and test-case-specific parameterization can yield meaningful performance gains, but identifying effective configurations requires extensive exploration, repeated testing, and careful analysis of the results over the tuning search space. The exploratory phase is then not merely an unavoidable overhead, but actually a source of rich performance data that, if systematically captured, can inform future configuration decisions. In this first section, we propose leveraging profiled data to construct surrogate models for an intent-driven middleware that enables users to express high-level deployment objectives rather than low-level configuration parameters.

By modeling application behavior from known executions, including the resources used and the parameters selected, users can express computational goals rather than low-level resource characteristics when launching jobs on an HPC system. In this setting, the proposed user-aware intent-driven middleware works to satisfy the Service-Level Objectives (SLOs) of an application, following the intent-driven principles described by Metsch et al. [45]. This approach treats execution traces as training data for application performance modeling and enables users to specify high-level objectives, properties, and constraints rather than manually configuring detailed parameters.

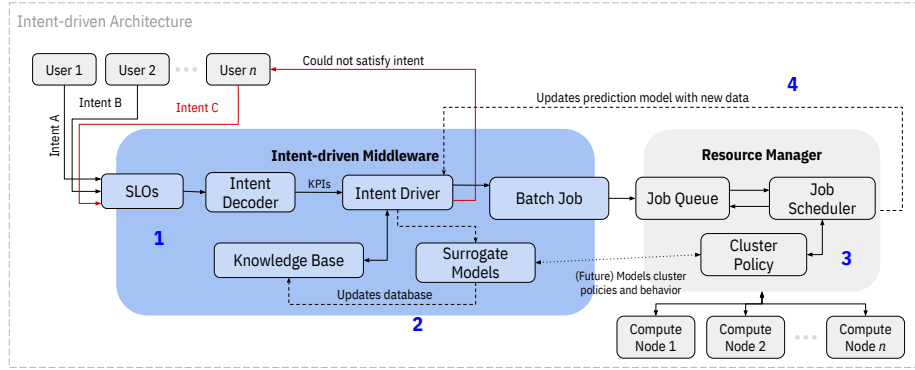
An intent may correspond to a runtime target, such as completing a simulation by a deadline, but it can also capture more complex goals. For example, if an application running on 10 nodes with 250 GB of memory per node encounters an out-of-memory condition, the user may wish to rerun it with a different configuration that avoids that failure mode. Rather than manually searching for such a configuration, the user can express that intent directly, allowing the system to determine whether a feasible configuration exists and, if so, identify the most appropriate one.

The middleware proposed here builds and maintains a surrogate performance model of the application running on an HPC infrastructure. By collecting traces of configuration-outcome pairs and using them to build a predictive performance model, the middleware can construct “what-if” scenarios to anticipate outcomes, reducing the need to incur costly executions. The ability to simulate a diverse set of scenarios in advance allows scientists to plan the execution of real experiments, avoid common pitfalls, reduce the number of costly experiments, and efficiently deploy complex configurations under infrastructural uncertainty. The end goal is to accelerate the test and validation of scientific workflows. As discussed in Section 1.1, much of the related work demonstrates the ability to model application behavior from execution traces, but still relies on an explicit job-scheduling model. We envision that redesigning the existing HPC scheduling architecture by introducing a new abstraction layer that translates high-level objectives into scheduler-consumable configurations can deliver substantial benefits for both users and system administrators.

Figure 4.1 contrasts the current job launching architecture (Figure 4.1a) with the proposed intent-driven alternative (Figure 4.1b). In the current architecture, users must manually specify low-level resource parameters when submitting jobs to the batch system (region 1). The resource manager then enqueues and schedules them according to cluster policy (regions 2 and 3) before dispatching work



(a) Current Architecture



(b) Intent-Driven Architecture

Figure 4.1: Comparison between current and intent-driven middleware architectures. The middleware serves as an additional layer between users and the resource manager, translating high-level user intents (SLOs) into job configurations and continuously refining recommendations based on execution feedback.

to compute nodes (region 4). The intent-driven middleware introduces an abstraction layer that allows domain scientists to express high-level objectives (SLOs, region 1), which are then decoded (Intent Decoder) into well-defined optimization problems at the Intent Driver component, solvable through constraint satisfaction or graph-based search strategies, such as the A* algorithm employed in [45]. The surrogate performance models (Surrogate Models) described above will serve as the knowledge base for the intent-driven middleware, informing configuration recommendations based on observed application behavior (region 2) and generating and submitting the batch job that performs the computing allocations (region 3). Crucially, the middleware should incorporate a feedback loop that continuously refines its surrogate models as new execution traces become available (region 4), improving the accuracy and reliability of its recommendations over time. When no feasible configuration can satisfy the expressed intent, the middleware returns a failure notification directly to the user rather than silently misconfiguring the job (red path).

To assess the feasibility of user-aware intent-driven middleware for scientific

workflows, we select WarpX as an initial test case. This choice is motivated by several factors: WarpX is a widely used application on supercomputing systems; prior work on a load-balancing toolbox for AMReX/WarpX has shown how difficult it is for users to identify effective balancing configurations; WarpX already produces tracing information during execution; and, because it is built on AMReX, insights from this case study may extend to other AMReX-based applications. Rather than starting from a hypothetical application model, this feasibility study asks whether the information generated by WarpX during routine development and testing can be used to construct a surrogate performance model to drive intent-aware middleware.

This preliminary study focuses on two measurable aspects of WarpX behavior as potential targets for computational intents: load-balancing efficiency and runtime. We have also considered node hours as a more generic metric. For the intents, we can target them individually or in tandem. For this initial proof-of-concept, we propose to use WarpX’s Laser-Ion Acceleration Plasma test case [10, 43, 69, 67], the same one used in the toolbox evaluation (Section 3.2).

A WarpX simulation is governed by several input categories that determine both the simulated physics and the computational cost: the domain geometry and mesh resolution, the time-stepping limit, the choice of physical model and solver, the boundary conditions for fields and particles, and the initialization of “species” and “sources”. Among these, the most influential for behavior and performance are the spatial discretization and refinement settings, since they directly control accuracy, memory use, and parallel workload; the solver and algorithm choices, which affect numerical stability and per-step cost; and the runtime controls such as moving-window, boosted-frame, and checkpoint/diagnostic settings, which shape overall execution efficiency. In practice, a WarpX input file specifies not just the problem to be solved, but also the numerical strategy and execution regime under which the simulation will run.

To isolate the effects of the most performance-sensitive inputs, we fix all but three configuration parameters in this study. Each execution uses a total grid size of 7488×14720 , advances for 2500 time steps, and adopts a minimum box size of 32×32 cells. We record load-balancing status every 100 iterations, using the *timers* heuristic to estimate the computational cost of each box. We vary the maximum box size, which in turn changes the total number of boxes and their distribution across the available GPUs (referred to as ranks, since the default configuration maps each MPI rank to a single GPU). We also vary the load-balancing algorithm and compare the Knapsack and SFC strategies. Each user-intent configuration is thus defined by a particular combination of maximum box size, GPU count, and load-balancing method.

We have a set of 45 real traces of this input set as a baseline, varying the maximum grid size and the number of parallel workers. Figure 4.2 depicts the search space spanned by the 45 real WarpX executions used in this study. Each point in the plot represents a real execution with a different combination of box count, number of GPUs, and load-balancing algorithm. We have collected traces from 10 different numbers of boxes, ranging from 32 to 780. From those, we tested different numbers of parallel workers within WarpX’s load-balancing

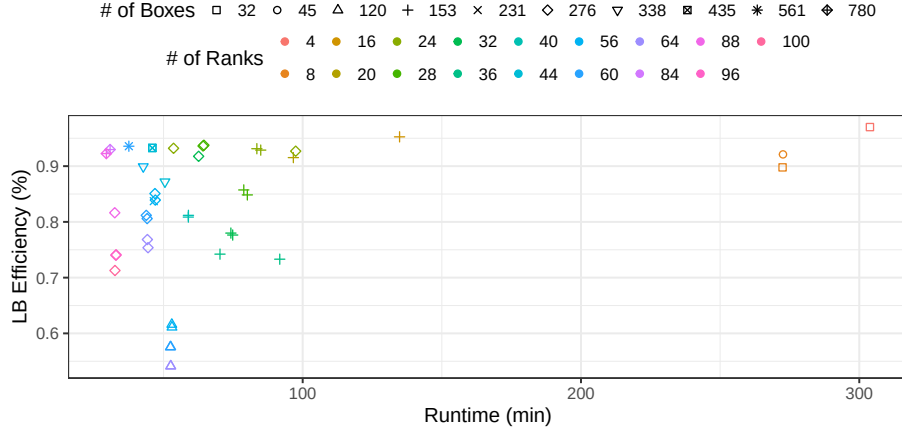


Figure 4.2: Search space of WarpX configurations across runtime and load-balancing efficiency used for this study.

delimitations. WarpX suggests that when running with load balancing enabled, the application should allocate at least 2 boxes per rank, and no more than 10-12 boxes per rank. We have run such a configuration on Perlmutter, the supercomputer system at the National Energy Research Scientific Computing Center. Perlmutter is an HPE Cray EX supercomputer with a heterogeneous architecture, comprising 3,072 CPU-only nodes and 1,792 GPU-accelerated nodes. The GPU partition is built around NVIDIA A100 Tensor Core GPUs, with GPU nodes typically equipped with four NVLink-connected A100s and a single AMD EPYC processor, providing a total of 6,159 GPUs across the system.

We observe that varying the box and GPU counts yields a diverse set of outcomes: load-balancing efficiency and runtime are not consistently correlated, with several configurations achieving high efficiency yet exhibiting widely varying runtimes, including outliers exceeding 250 minutes. The presence of clearly suboptimal configurations in the space underscores the cost of uninformed exploration: without guidance, a domain scientist could easily submit jobs that land in the worst-performing regions of this space.

We then formulate a composite surrogate modeling framework for this input set, consisting of two main stages: first, estimate the load-balancing efficiency using the AMReX-based load balance toolbox; and second, use that estimate to predict runtime. When no prior information is available for a new maximum box size, an initial baseline stage is required to characterize the workload distribution, since changes in box size alter the load distribution and, consequently, the load-balance efficiency. In this formulation, load-balance modeling complements runtime modeling by providing an intermediate description of how work is actually distributed inside the application. The following sections discuss and validate each step of the framework in depth.

4.1.1 Cluster resampling

Load balancing efficiency is closely related to the distribution of work performed in parallel and the number of available parallel workers. A WarpX grid can be partitioned into a varying number of boxes, depending on the maximum box size specified for a given execution. Given this size, the adaptive mesh refinement strategy implemented by the AMReX framework will produce different box counts depending on the strategies chosen. This process involves complex mathematical functions that target higher resolution within the grid's region of interest, where the actual physical simulation takes place. Throughout the simulation, the AMReX strategy can choose to subdivide cells into fewer or more boxes as needed, dynamically redistributing work across MPI ranks.

A key challenge in building a surrogate performance model is that profiling every possible configuration is prohibitively expensive. Ideally, a single profiled execution should be sufficient to reason about the behavior of unseen configurations with different maximum box sizes and box counts. However, each configuration produces a different workload distribution, and directly reusing the weights from one execution for another would be inaccurate. For load-balancing purposes, the efficiency of work distribution does not necessarily depend on the exact weight values obtained in a specific execution, but rather on accurately capturing the shape and relative distribution of weights. Given the dynamic nature of the AMR breaking from AMReX, this is a challenging task.

To generate synthetic workload distributions that resemble those observed in real runs, we explored several rescaling strategies and identified cluster resam-

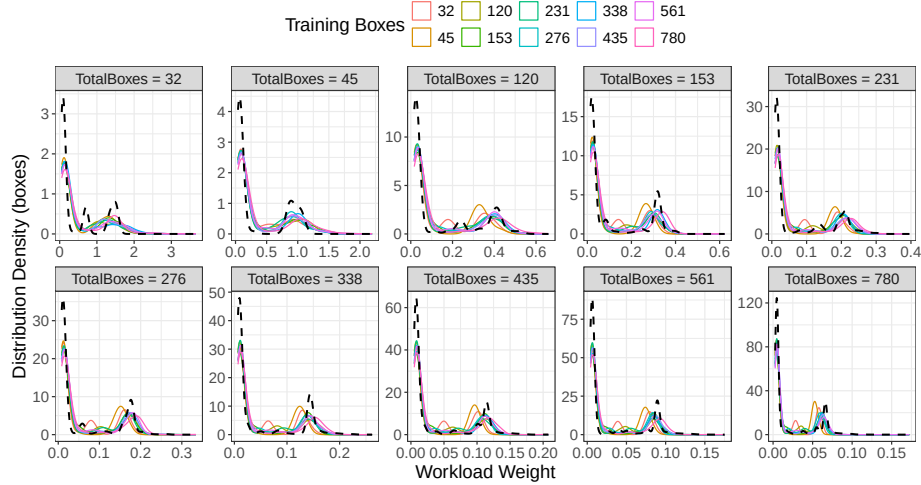


Figure 4.3: Distribution density comparison between the real weight distribution (black dashed curve) and distributions made by the cluster resampling method using different baselines (colored curves). Each training distribution is identified with a different color.

pling as the most promising approach. We generate synthetic workload distributions by rescaling a baseline set of box weights to a target number of boxes while preserving the total workload. The procedure applies k -means clustering [42] to capture the structural patterns in the rescaled weights, then resamples from the resulting clusters with controlled noise to produce synthetic distributions that are representative of realistic workloads. The final distribution is clipped to valid bounds and renormalized to preserve the total workload. This allows a single profiled execution to serve as the basis for estimating workload distributions across a wide range of unseen configurations, enabling the surrogate model to generalize beyond the configurations that have been explicitly run.

As shown in Figure 4.3, for 7 of the 10 configurations explored (Total Boxes = 45, 231, 276, 338, 435, 561, 780), the cluster resampling strategy yields synthetic distributions that are in close agreement with the reference. However, for configurations where the total number of boxes is 32, 120, and 153, the resulting shapes exhibit considerable differences, directly linked to the previously explained AMR grid decomposition. Configurations that produce similar patterns likely decompose the grid into comparable structures, whereas those for which the cluster resampling method performs poorly involve fundamentally different decompositions.

We quantify distribution similarity using two complementary measures: the Hellinger distance [7] and the Wasserstein-1 distance, also known as the Earth Mover’s Distance (EMD) [65]. The Hellinger distance equals 0 for identical distributions and 1 for completely disjoint ones, while the EMD quantifies the amount of mass redistribution required to transform a synthetic distribution into the reference. Across all configurations, we observe a mean Hellinger distance of 0.51, with the best individual pairs reaching $H = 0.43$, indicating substantial distributional overlap, particularly for mid-to-large configurations. The normalized EMD shows a transport cost below 0.20 for configurations that share similar AMR decomposition behavior, whereas configurations with structurally different decompositions — such as those producing 32 or 45 total boxes — exhibit higher transport costs (≈ 0.26 and ≈ 0.28 , respectively), reflecting the structural difficulty discussed above. Addressing these cases by accounting for differences in AMR decomposition to better predict load distributions across configurations remains an open challenge and a direction for future work.

4.1.2 Surrogate Runtime Model

Having established a method for generating synthetic workload distributions, we turn to the final modeling step: estimating the runtime of unseen configurations. We train a regression model to estimate the runtime of an iteration from three inputs: the number of boxes used by the simulation, the number of parallel workers, and the load-balancing efficiency for that iteration. We employ XGBoost (Extreme Gradient Boosting) [14] as the regression strategy because the relationships among load-balancing efficiency, parallelism, and runtime are likely nonlinear. Additionally, gradient-boosted trees are known to generalize effectively from relatively small training sets, which is a realistic constraint given

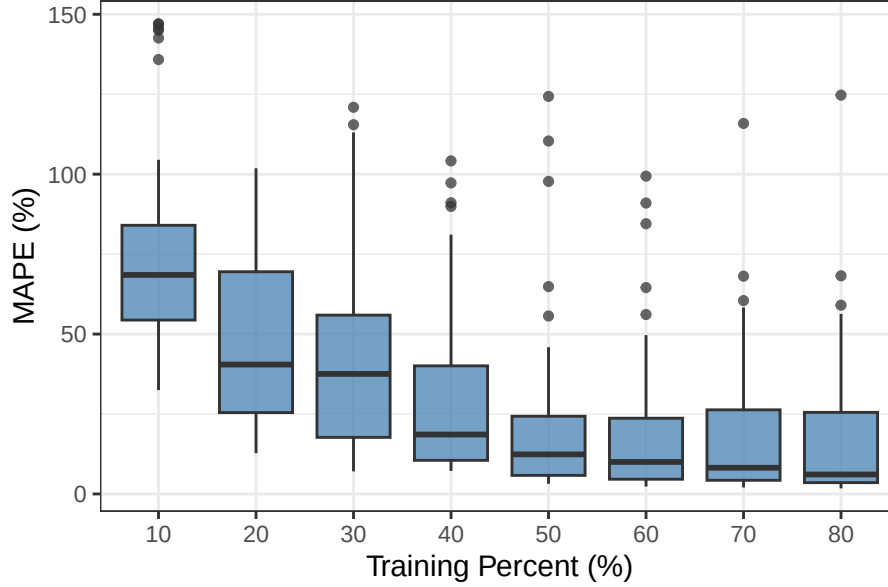


Figure 4.4: XGBoost runtime prediction error across training data sizes. Each boxplot summarizes the Mean Absolute Percentage Error (MAPE) across 50 random train-test partitions for each training percentage, aggregated over the 10 grid configurations.

the cost of collecting execution traces.

Figure 4.4 shows how the Mean Absolute Percentage Error (MAPE) evolves as the training data size increases from 10% to 80% of the available dataset, comprising 45 executions in total. Each boxplot summarizes the MAPE for runtime prediction across the 10 grid configurations and varying GPU ranks, aggregated over 50 random train-test partitions at each training percentage to account for sampling variability.

A clear monotonic decrease in both median MAPE and interquartile range is observed as the training proportion increases, demonstrating that model predictions improve substantially and stabilize with greater data availability. At 10% training data (corresponding to approximately 5 executions), the median MAPE exceeds 60%, with considerable spread and outliers, indicating poor predictions. Error declines across the 20%–40% percentages, with median MAPE falling to approximately 35%–40% before plateauing between 10%–20% for training percentage of 50% and above. The distribution narrows, despite still presenting outliers, when using more than 23 different executions to train the model, suggesting good convergence. Such high-magnitude outliers can be attributed to the complexity of AMR grid partitioning discussed in the cluster resampling section and to the smaller number of training data for bigger maximum grid sizes (which generate fewer boxes). Load-balancing efficiency for such samples

can still yield good results, but using fewer parallel workers increases runtime. This aspect, combined with the AMR grid partitioning, can lead to different loads to the system, resulting in different runtime behavior, and more sampling is needed to accurately predict performance for those cases.

4.1.3 Intent-Driven Configuration Selection

With the three modeling components in place (workload distribution estimation, load-balancing efficiency prediction, and runtime regression), we now combine them into a unified intent-driven system. To begin generating predictions, the system requires at least one initial execution trace. As additional traces become available, the model is refined, and its configuration suggestions more reliably fulfill the user’s intent. Each time a new execution is completed, the system updates its models and replaces synthetic simulation data with real execution data.

4.1.3.1 Exploring a Runtime Intent

We now present a concrete example of intent-driven configuration selection. Consider a simple request: “Execute this input set in under 50 minutes”. Figure 4.5 shows the results, where the x-axis represents the iteration of the intent-resolution process (which also corresponds to the number of execution traces used for modeling at that point), and the y-axis shows the total runtime, in minutes, of the configuration selected at each iteration. The baseline execution, in which the grid was divided into 276 boxes, was run on 24 GPUs in ≈ 97 minutes.

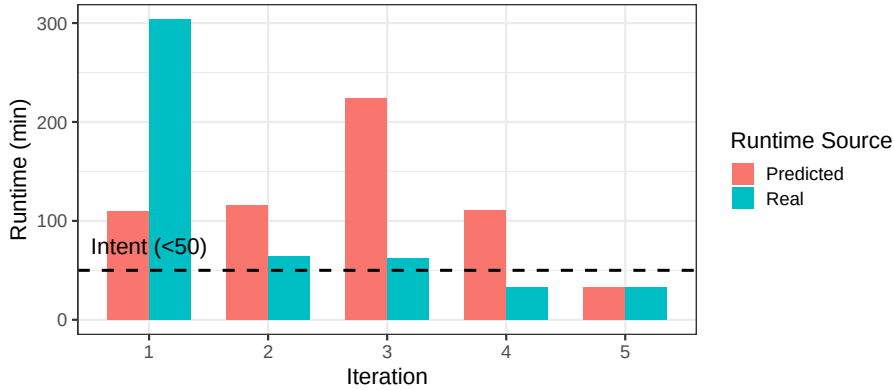


Figure 4.5: The process until the intent achievement. After an initial execution, the system does not find any solution that meets the user’s intent and instead runs random predictions. After a few iterations, a suggested run that was not predicted achieves the user’s intent, so in the next iteration, the system will suggest a correct configuration.

The system will then follow a constraint-driven selection policy, considering only configurations with predicted runtimes below 50 minutes as viable candidates. If such a configuration exists, the policy selects and recommends it. When no configuration is predicted to satisfy the user’s intent, the policy proposes an unexplored configuration instead. The first trial produces a significantly inaccurate prediction, with the observed runtime nearly $3\times$ higher than the estimate. Although incorrect, this outcome provides new information that is incorporated into subsequent model updates. As a result, later predictions better reflect observed behavior. However, early iterations still fail to meet the desired constraint, even as prediction accuracy improves. At iteration 4, despite not predicting it, the system runs a configuration with a runtime of less than 50 minutes. Incorporating that result, the system can then suggest such a configuration on the next iteration and achieve the user’s intent.

Different intents may require different numbers of iterations to find a solution. Not only that, but different starting points and information in the intent system can also drive different results. Figure 4.6 depicts the average number of iterations and average cost, in node hours, to achieve different runtime intents using different configurations as a starting point. For this analysis, we consider the set of 45 different real runs as a baseline. On the x-axis, we have the different intents tested, varying from less than 30 minutes to less than 2 hours. These intents were chosen based on the runtime distribution of our data sample. Then we present the average cost, in node-hours, to achieve the different intents in the left plot, while the right plot shows the average number of iterations on the system to achieve each intent. Node cost was measured using Perlmutter’s charge cost, described in Chapter 2.

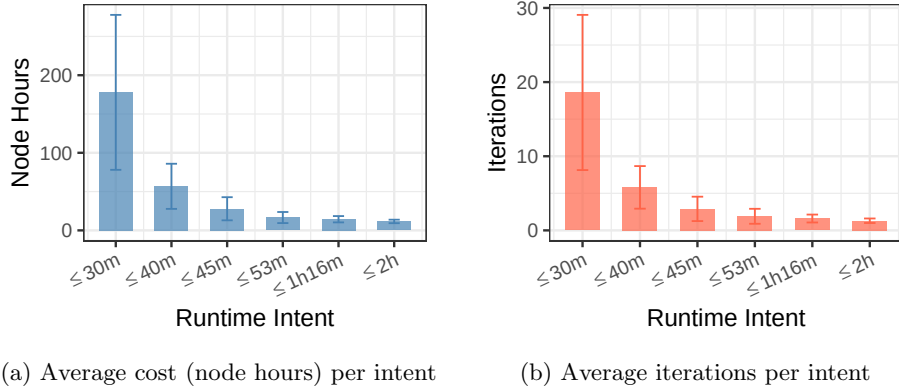


Figure 4.6: Average cost and average number of iterations to achieve different runtime intents. Figure 4.6a shows the average number of node-hours spent to achieve the intent, and Figure 4.6b shows the number of iterations required.

The runtime intents were defined by selecting the quartiles of the runtime distribution observed across the 45 real executions used for validation. The distance from the average to the extremities of the error bars was calculated

using a 95% confidence interval, i.e., $\frac{2\sigma}{\sqrt{N}}$, where σ represents the standard deviation, and N the number of measurements. As expected, the two plots exhibit similar trends because node-hour cost is primarily driven by the number of iterations required to find a satisfactory configuration. This confirms that reducing the number of exploration iterations is the key lever for minimizing the computational overhead of the intent-resolution process.

For relaxed intents, where the runtime can be as little as 45 minutes and as long as 2 hours, the system can find a solution in an average of 1 to 3 iterations, costing about 11 to 16 node-hours, with narrow error bars indicating reliable convergence regardless of the starting configuration. As the runtime constraint tightens, both iteration count and cost increase. To run with less than 30 minutes, the results are qualitatively different from the rest, since it is a regime in which few configurations satisfy it. Despite having a higher average, our surrogate-guided search increasingly gives way to exploratory behavior using less than half of the available data.

4.1.3.2 Exploring a Load Balance Intent

We perform a similar case study, now focusing on load-balancing efficiency as the target intent. Here, the goal is to achieve an overall load-balancing efficiency of at least 93%. Figure 4.7 depicts the results, with the x-axis showing the iterations required to achieve the intent and the y-axis showing the load-balancing efficiency. In this scenario, we start our execution with a configuration that divides the grid into 120 boxes and run it with 56 ranks, yielding an overall load-balancing efficiency of $\approx 62\%$.

We perform a similar demonstration, now focusing on load-balancing efficiency as the target intent. Here, the goal is to achieve an overall load balance efficiency of at least 93%. The intent threshold of 93% was chosen to reflect load-balancing efficiency observed across the profiled executions in this study, ensur-

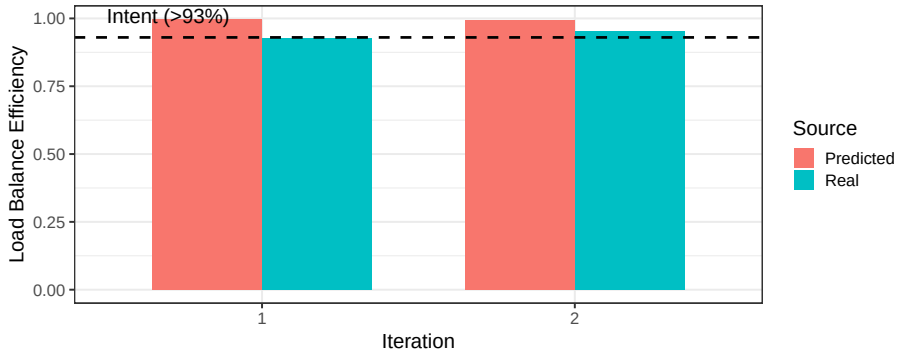


Figure 4.7: Process until the load balance efficiency intent achievement. After the initial execution, the system estimates the workload distribution for new grid breaks and projects their load-balance efficiency.

ing that the target is grounded in realistic, achievable system behavior. From the baseline configuration, which achieves only $\approx 62\%$ efficiency, the system must identify configurations that meet this threshold through surrogate-guided exploration.

In this case, the intent can be achieved with fewer iterations than for runtime. Starting with the workload distribution for 120 boxes, we apply our cluster resampling strategy to redistribute the workload across the remaining 9 configurations used in this investigation and predict their load-balancing efficiency. In the first iteration, the system identifies a configuration that it predicts will exceed the intent threshold, though the predicted efficiency exceeds what the actual execution achieves. The system then updates its surrogate model with the observed result and proposes a new configuration in the second iteration, which again satisfies the intent. Notably, both iterations exhibit a consistent pattern of overprediction, where the surrogate model overestimates load-balancing efficiency relative to the real execution. This behavior is consistent with the model limitations discussed in Section 4.1.1, and suggests that while the model is sufficient to guide intent resolution, improving prediction accuracy remains an important direction for future work.

We have also investigated the average number of iterations and the cost of finding load-balancing intents. Figure 4.8 demonstrates the results. Again, the two plots exhibit similar trends because node-hour cost is primarily driven by the number of iterations required to find a satisfactory configuration. Finding load-balancing efficiency is considerably easier than runtime efficiency, since it depends only on the execution’s load distribution. By leveraging cluster resampling and combining it with the AMReX load balancing toolbox, we obtain accurate estimates of the workload distribution without requiring exhaustive sampling across all configurations, even though this technique does not uniformly dominate in terms of accuracy. Furthermore, this study explores an exclusive input set of WarpX. When extending this approach to other input sets and targeting different load-balancing efficiencies, this cost-to-intent will probably exhibit different behavior.

4.1.3.3 Exploring Complex Intents

We now demonstrate how this preliminary intent system can achieve more composed and complex intents. As discussed in Chapter 2, most supercomputing facilities allocate a fixed number of node-hours budget per project. Consequently, users often seek to control execution costs while also achieving their service-level objectives on each run. For instance, a user may request a configuration that completes in under 50 minutes and ensures the total cost does not exceed 12 node-hours. Figure 4.9 illustrates how the intent system responds to this composed objective.

In this demonstration, satisfying the node-hours intent alone is insufficient; the runtime constraint must also be met. When multiple intents are specified, the system must find a configuration that jointly satisfies all constraints. This can be observed in the plots, where, although the node-hours intent is met in the

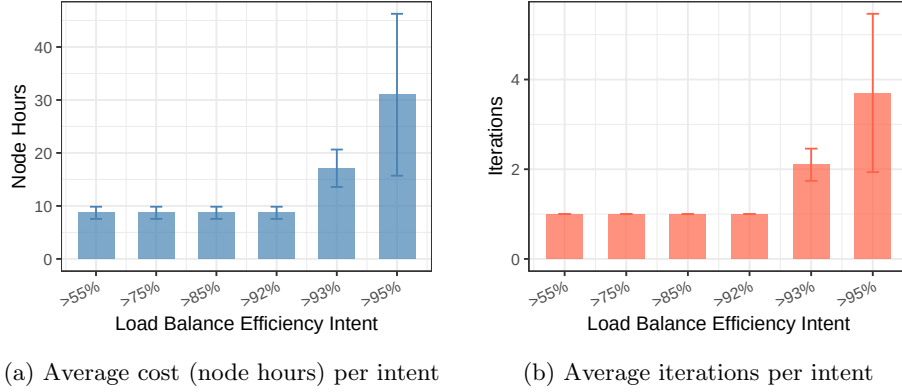


Figure 4.8: Average cost, in node hours, and iterations, to achieve a load balance efficiency intent.

first iteration (Figure 4.9b), the runtime is not achieved, so the system continues exploring configurations. Such a configuration is found at iteration 8, satisfying the user’s intent with a runtime of 45 minutes and a cost of 11 node-hours.

Our last example extends the previous composed intent to consider load-balancing efficiency as well. In this case, the system needs to find a configuration that runs in less than 50 minutes, uses no more than 12 node hours, and achieves a load-balancing efficiency of at least 93%. As shown in Figure

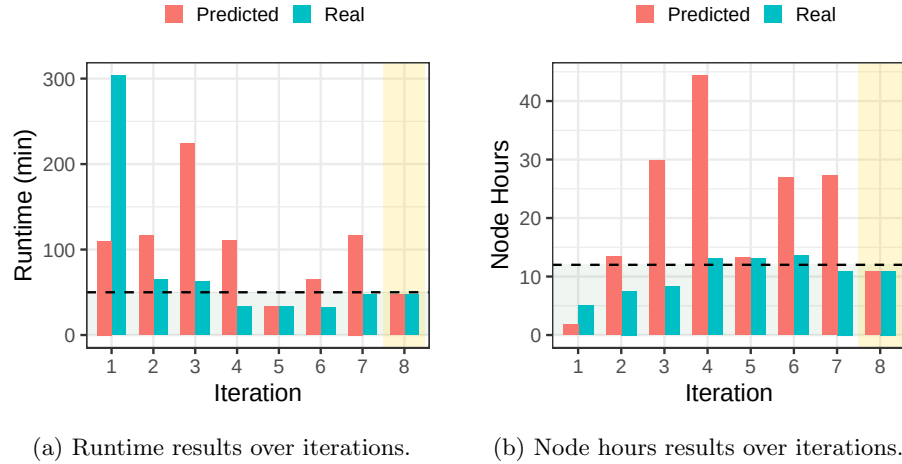


Figure 4.9: Intent system behavior for a composed two-objective intent. Dashed lines indicate the runtime intent thresholds: under 50 minutes and at most 12 node-hours. The highlighted iteration finds the configuration that meets both constraints.

4.10, over iterations, different configurations achieve each intent separately, but only in iteration 13 does the system find a configuration that simultaneously achieves the composed intent. Unlike single-objective intents, where the system searches for any configuration that meets a single threshold, composed intents impose a joint constraint whose feasible region is harder to know a priori and may be small. Additionally, the three constraints may not be independent, as in this example. Increasing GPU count tends to reduce runtime but can increase node-hour cost, and the combination of the number of boxes and the number of GPUs is non-linear with respect to load-balance efficiency. A configuration that satisfies any two of the three constraints may still violate the third, so the system must identify a configuration in which all three thresholds are cleared simultaneously.

These initial results demonstrate the promise of our proposed user-aware intent-driven system for HPC applications. By composing models that reason about user intents, this preliminary study effectively identifies configurations without exhaustively exploring the full 45-candidate execution space. Crucially, the system’s goal is not to find a globally optimal solution, but rather to identify a configuration that reliably satisfies the user’s intent. This distinction makes the approach practical for real-world job submission scenarios where users lack deep systems expertise.

The next steps of this research include exploring more principled techniques for navigating the configuration search space, such as reinforcement learning, which offers a natural framework for balancing exploration of unseen configurations against exploitation of known effective ones. We will extend this study to other applications, such as VG Giraffe, and demonstrate how this approach can enable intent-driven execution in the face of the complexity of tunner parameters observed in the miniGiraffe exploration. Defining terms and notation, translating users’ intent into KPIs, and investigating sophisticated strategies to reduce the number of iterations are also part of the next steps.

4.2 Cluster-aware intent-driven system

In the first step of our HPC intent-driven approach, the middleware serves as a front-end, operating independently of the job scheduler and generating conventional job descriptions using the parameters optimized to satisfy the user’s intent. From the resource manager’s perspective, these submissions are indistinguishable from any conventional job. In this second section, we plan to incorporate cluster awareness into the intent-driven system and embed intent information directly into scheduling policies via existing extension interfaces, such as scheduler plugins.

We plan to integrate with and extend existing HPC schedulers, such as Slurm, PBS, and Flux, rather than replace them, by defining a strategy for incorporating intent constraints into their scheduling logic. These production resource managers are already well established and expose low-level interfaces suitable for extension and adaptation. By positioning intent-driven orchestration as

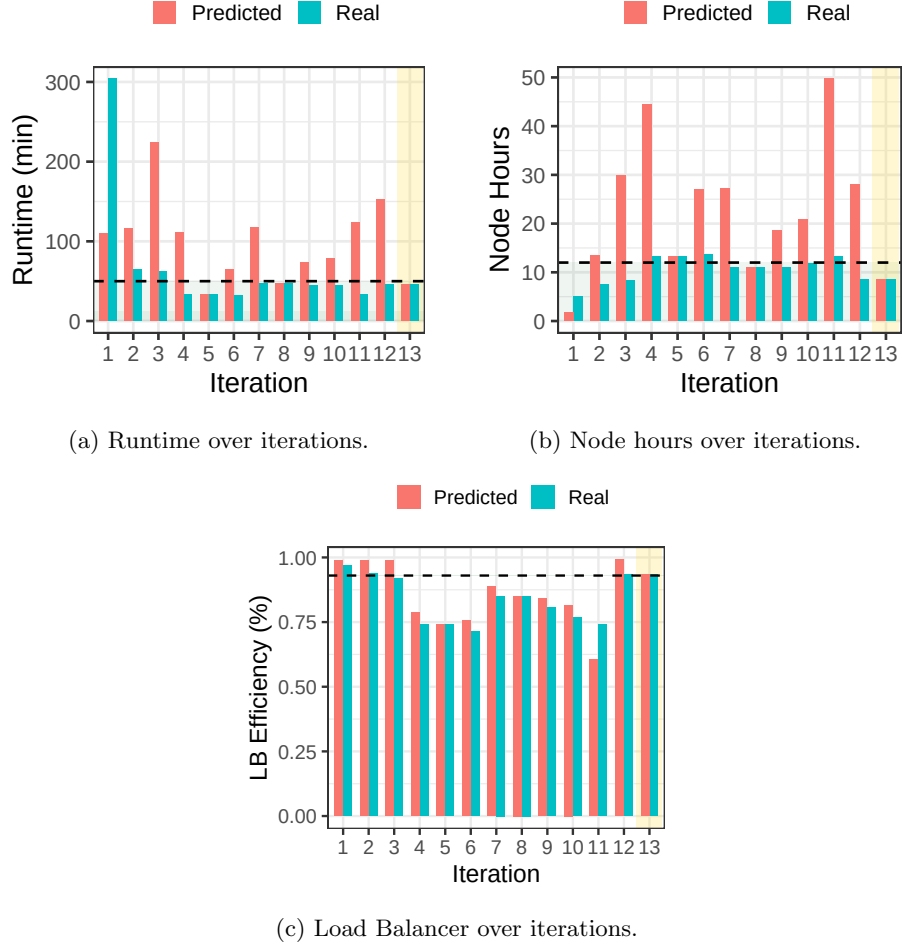


Figure 4.10: Intent system behavior under a composed three-objective constraint. Dashed lines indicate the intent thresholds: under 50 minutes, at most 12 node hours, and at least 93% LB efficiency. The highlighted iteration finds the configuration that meets all three constraints.

an additional layer that translates high-level intents into scheduler-consumable configurations and policies, we can leverage the maturity of these systems while introducing new capabilities centered on user goals and application behavior.

In this new scenario, the resource orchestrator must reason about multiple simultaneous intents similarly to how current systems handle multiple job queues. When the intent layer finds configurations that satisfy a user’s intent for a given application, a natural next step is to check whether the resources needed by the chosen configuration are available under current conditions, and to adjust the recommendation if not, all while maximizing the user’s SLOs. This requires adapting and defining the chosen configurations to also account for schedulers’ metadata, such as predicted slowdown, deadline satisfaction, energy per job or campaign, and maximum or preferred core counts, which will be derived from a combination of user input, application profiling, and surrogates. We can then explore how these measurements can be incorporated into the scheduler’s scoring functions or backfilling heuristics, alongside the current factors considered, such as estimated wall time, requested node count, and queue priority. Intent metadata, application predictions, and system-state information can be integrated within scheduler plugins or policy extensions to trade off runtime and resource usage in a principled way. This is conceptually analogous to how related work incorporates workflow structure and I/O demands into scheduling decisions [17], but with the distinguishing characteristic that the optimization objectives are derived explicitly from user-specified intents rather than from system-level heuristics. However, moving from intent-aware recommendations to autonomous execution introduces additional challenges.

Rather than only suggesting configurations, the intent-driven system will also take actions on the users’ behalf. This shift can introduce a new class of challenges beyond those in the user-only phase, such as handling potentially conflicting intents and matching users’ intents to system policies. For instance, a user can provide a logically contradictory intent: they may want to improve communication locality while simultaneously asking to distribute the workload across heterogeneous node types that are physically apart in the cluster. The orchestrator must detect such logical inconsistencies and either reject the request, request clarification, or propose alternative configurations that approximate the user’s intent while remaining within system constraints.

Another challenge concerns the latency in scheduling decisions, which can arise from the search for configurations that satisfy complex intents. Supercomputing facilities have a fast-moving cluster state, since they can receive thousands of job submissions per minute. Resource availability can change rapidly between when a user (or service) queries system status and when a job is finally scheduled, potentially degrading SLO satisfaction unless the system accounts for this drift in its decision-making process.

To mitigate this source of decision latency and state drift, we argue that accurate surrogate models will be needed at both the application and cluster levels. A cluster-level surrogate model will serve as an important source of information for decision-making, enabling the system to identify which actions can jointly satisfy user-specified intents and facility-level policies (e.g., fairness,

utilization, and energy constraints) without repeatedly querying the live system at fine temporal granularity. This idea is motivated by works such as [28, 2], which apply machine learning techniques to scheduling optimization, addressing the limitations of conventional scheduling approaches and supporting multi-objective goals encompassing energy efficiency, performance, and cost.

Finally, balancing conflicting goals, such as minimizing runtime while reducing power consumption or cost, is challenging because improving one objective typically degrades another. Developing intent-aware policies that explicitly represent, explore, and communicate these trade-offs is therefore a key part of the proposed work. If successful, such policies can lead to significant improvements in resource use and user experience by making the compromises between performance, energy, and fairness transparent and tunable, rather than implicit in fixed scheduler heuristics.

Chapter 5

Conclusion

The hardware and software heterogeneity present in modern supercomputing facilities enables higher performance and greater flexibility across diverse workload types, but also poses significant challenges for end users in identifying configurations that meet their computational objectives. The integration of AI-driven and data-informed strategies to support scientific discovery has gained increasing attention, as evidenced by recent academic work. This reflects a broader transition in the HPC landscape, where emerging workloads and workflows place new demands on existing infrastructure and necessitate more advanced orchestration strategies.

My work on miniGiraffe and the AMReX Load Balancing Toolbox demonstrates research contributions that I have already made in performance characterization and optimization for scientific applications. Both results are publicly available and enable exploration of hardware and tuning for new types of workloads (with the miniGiraffe proxy) and of workload distribution (for the load balancing toolbox). Building on these foundations, this proposal details two projects that ultimately lead to a novel intent-driven orchestration system for scientific computing. We provide a detailed discussion of how a user-aware, intent-driven system can help users identify configurations that achieve their computational intents. The first describes user-aware, intent-driven middleware and includes a feasibility study demonstrating its potential, presents initial developments and results, and outlines the future steps required to generalize the solution across applications. The second project presents a cluster-aware, intent-driven orchestrator, in which we aim to extend existing resource managers to incorporate user-specific.

Bibliography

- [1] Omar Aaziz, Jeanine Cook, Jonathan Cook, Tanner Juedeman, David Richards, and Courtenay Vaughan. A methodology for characterizing the correspondence between real and proxy applications. In *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 190–200, 2018.
- [2] Kyrian Adimora, Sai Rithvik Gundla, and Hongyang Sun. Machine learning approaches for optimizing high-performance computing scheduling: a comprehensive survey and analysis. *Cluster Computing*, 28(13):831, 2025.
- [3] Negin Akbari, John Grundy, Aamir Cheema, and Adel N Toosi. Intentcontinuum: Using llms to support intent-based computing across the compute continuum. *arXiv preprint arXiv:2504.04429*, 2025.
- [4] Negin Akbari, Adel N. Toosi, John Grundy, Hourieh Khalajzadeh, Mohammad S. Aslanpour, and Shashikant Ilager. iContinuum: An emulation toolkit for intent-based computing across the edge-to-cloud continuum. In *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*, pages 468–473. IEEE, 2024.
- [5] Scott Atchley, Rosa M. Badia, Bronis R. de Supinski, Joshua Fryman, Dieter Kranzlmüller, Srilatha Manne, Pekka Manninen, Satoshi Matsuoka, Dejan Milojicic, Galen Shipman, Eric Van Hensbergen, and Robert W. Wisniewski. Predicting the future of supercomputing. *Computer*, 58(7):110–120, 2025.
- [6] Ilyes Batatia, David P Kovacs, Gregor Simm, Christoph Ortner, and Gabor Csanyi. Mace: Higher order equivariant message passing neural networks for fast and accurate force fields. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 11423–11436. Curran Associates, Inc., 2022.
- [7] Rudolf Beran. Minimum hellinger distance estimates for parametric models. *The annals of Statistics*, pages 445–463, 1977.

- [8] Josep Lluís Berral, David Aguilera-Luzón, Peini Liu, Ramon Nou, Maria Serrano, Angelos Antonopoulos, Javier Santaella, and Mario José Diván. Integrating reliability into intent-driven orchestration on kubernetes-based edge nodes. In *2025 IEEE 33rd International Conference on Network Protocols (ICNP)*, pages 1–6, 2025.
- [9] Mark Bohr. A 30 year retrospective on dennard’s mosfet scaling paper. *IEEE Solid-State Circuits Society Newsletter*, 12(1):11–13, 2007.
- [10] S. S. Bulanov, A. Brantov, V. Yu. Bychenkov, V. Chvykov, G. Kalinchenko, T. Matsuoka, P. Rousseau, S. Reed, V. Yanovsky, D. W. Litzenberg, K. Krushelnick, and A. Maksimchuk. Accelerating monoenergetic protons from ultrathin foils by flat-top laser pulses in the directed-coulomb-explosion regime. *Phys. Rev. E*, 78:026412, Aug 2008.
- [11] Onur Cankur, Brian Austin, and Abhinav Bhatele. Understanding gpu utilization using ldms data on perlmutter. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC’25 Posters)*. ACM/IEEE, 2025.
- [12] Cveta Capova. Intent translation for computing continuum management through knowledge-graph-enabled agentic reasoning. Diploma thesis, Technische Universität Wien, 2025. [repositUM](#).
- [13] Emanuele Carlini, Massimo Coppola, Patrizio Dazzi, Luca Ferrucci, Hanna Kavalionak, Ioannis Korontanis, Matteo Mordacchini, and Konstantinos Tserpes. Smartorc: smart orchestration of resources in the compute continuum. *Frontiers in High Performance Computing*, Volume 1 - 2023, 2023.
- [14] Tianqi Chen, Tong He, Michael Benesty, Vadim Khotilovich, Yuan Tang, Hyunsu Cho, Kailong Chen, Rory Mitchell, Ignacio Cano, Tianyi Zhou, et al. Xgboost: extreme gradient boosting. *R package version 0.4-2*, 1(4):1–4, 2015.
- [15] Václav Chlumský and Dalibor Klusáček. Improving accuracy of walltime estimates in pbs professional using soft walltimes. In *Job Scheduling Strategies for Parallel Processing: 25th International Workshop, JSSPP 2022, Virtual Event, June 3, 2022, Revised Selected Papers*, page 192–210, Berlin, Heidelberg, 2022. Springer-Verlag.
- [16] Jessica I Dagostini, Joseph B Manzano, Tyler Sorensen, and Scott Beamer. minigiraffe: A pangenomic mapping proxy app. In *2025 IEEE International Symposium on Workload Characterization (IISWC)*, pages 339–352. IEEE, 2025.
- [17] Yiqin Dai, Ruibo Wang, Yong Dong, and Kai Lu. Towards highly compatible i/o-aware workflow scheduling on hpc systems. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2024.

- [18] Tom Deakin, Simon McIntosh-Smith, James Price, Andrei Poenaru, Patrick Atkinson, Codrin Popa, and Justin Salmon. Performance portability across diverse computer architectures. In *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, pages 1–13. IEEE, 2019.
- [19] Jordan M. Eizenga, Adam M. Novak, Jonas A. Sibbesen, Simon Heumos, Ali Ghaffaari, Glenn Hickey, Xian Chang, Josiah D. Seaman, Robin Rounthwaite, Jana Ebler, Mikko Rautiainen, Shilpa Garg, Benedict Paten, Tobias Marschall, Jouni Sirén, and Erik Garrison. Pangenome graphs. *Annual Review of Genomics and Human Genetics*, 21(1):139–162, 2020.
- [20] Yuping Fan, Paul Rich, William E Allcock, Michael E Papka, and Zhiling Lan. Trade-off between prediction accuracy and underestimation rate in job runtime estimates. In *2017 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 530–540. IEEE, 2017.
- [21] Luca Fedeli, Axel Huebl, France Boillod-Cerneux, Thomas Clark, Kevin Gott, Conrad Hillairet, Stephan Jaure, Adrien Leblanc, Rémi Lehe, Andrew Myers, Christelle Piechurski, Mitsuhsa Sato, Neïl Zaim, Weiqun Zhang, Jean-Luc Vay, and Henri Vincenti. Pushing the frontier in the design of laser-based electron accelerators with groundbreaking mesh-refined particle-in-cell simulations on exascale-class supercomputers. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12, 2022.
- [22] Nikos Filinis, Ioannis Tzanettis, Dimitrios Spatharakis, Eleni Fotopoulou, Ioannis Dimolitsas, Anastasios Zafeiropoulos, Constantinos Vassilakis, and Symeon Papavassiliou. Intent-driven orchestration of serverless applications in the computing continuum. *Future Generation Computer Systems*, 154:72–86, 2024.
- [23] Erik Garrison, Jouni Sirén, Adam M Novak, Glenn Hickey, Jordan M Eizenga, Eric T Dawson, William Jones, Shilpa Garg, Charles Markello, Michael F Lin, et al. Variation graph toolkit improves read mapping by representing genetic variation in the reference. *Nature biotechnology*, 36(9):875–879, 2018.
- [24] Markus Geimer, Felix Wolf, Brian JN Wylie, Erika Ábrahám, Daniel Becker, and Bernd Mohr. The scalasca performance toolset architecture. *Concurrency and computation: Practice and experience*, 22(6):702–719, 2010.
- [25] Robert L Henderson. Job scheduling under the portable batch system. In *Workshop on Job Scheduling Strategies for Parallel Processing*, pages 279–294. Springer, 1995.
- [26] John L Hennessy and David A Patterson. A new golden age for computer architecture. *Communications of the ACM*, 62(2):48–60, 2019.

- [27] John L. Hennessy, David A. Patterson, and Christos Kozyrakis. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, San Francisco, CA, 7th edition, 2025.
- [28] Zhengxiong Hou, Hong Shen, Xingshe Zhou, Jianhua Gu, Yunlan Wang, and Tianhai Zhao. Prediction of job characteristics for intelligent resource allocation in hpc systems: a survey and future directions. *Frontiers of Computer Science*, 16(5):165107, 2022.
- [29] International Energy Agency (IEA). Energy and AI. Technical report, IEA, Paris, 2025. Licence: CC BY 4.0.
- [30] Morris A. Jette and Tim Wickberg. Architecture of the slurm workload manager. In *Job Scheduling Strategies for Parallel Processing: 26th Workshop, JSSPP 2023, St. Petersburg, FL, USA, May 19, 2023, Revised Selected Papers*, page 3–23, Berlin, Heidelberg, 2023. Springer-Verlag.
- [31] Ian Karlin, Abhinav Bhatele, Jeff Keasler, Bradford L. Chamberlain, Jonathan Cohen, Zachary Devito, Riyaz Haque, Dan Laney, Edward Luke, Felix Wang, David Richards, Martin Schulz, and Charles H. Still. Exploring traditional and emerging parallel programming models using a proxy application. In *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*, pages 919–932, 2013.
- [32] Narendra Karmarkar and Richard M Karp. *The differencing method of set partitioning*. Computer Science Division (EECS), University of California Berkeley, 1982.
- [33] Aditya Kashi, Hao Lu, Wesley Brewer, David Rogers, Michael Matheson, Mallikarjun Shankar, and Feiyi Wang. Mixed-precision numerics in scientific applications: survey and perspectives. *The Journal of Supercomputing*, 82(5):287, 2026.
- [34] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.
- [35] Awais Khan, Hyogi Sim, Sudharshan S. Vazhkudai, Ali R. Butt, and Youngjae Kim. An analysis of system balance and architectural trends based on top500 supercomputers. In *The International Conference on High Performance Computing in Asia-Pacific Region, HPCAsia '21*, page 11–22, New York, NY, USA, 2021. Association for Computing Machinery.
- [36] Dalibor Klusáček and Václav Chlumský. Fair-sharing simulator: Toward fair scheduling in batch computing systems. *The International Journal of High Performance Computing Applications*, 40(2):254–270, 2026.

- [37] Jakub Kudela and Radomil Matousek. Recent advances and applications of surrogate models for finite element method computations: a review. *Soft Computing*, 26:13709–13733, 2022.
- [38] Maria Daniela Leite de Souza and Thijs Metsch. Advancing orchestration: Leveraging distance functions to meet application intents. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*, pages 82–87, 2024.
- [39] Aris Leivadeas and Matthias Falkner. A survey on intent-based networking. *IEEE Communications Surveys & Tutorials*, 25(1):625–655, 2023.
- [40] Jie Li, George Michelogiannakis, Brandon Cook, Dulanya Cooray, and Yong Chen. Analyzing Resource Utilization in an HPC System: A Case Study of NERSC’s Perlmutter. In Abhinav Bhatele, Jeff Hammond, Marc Baboulin, and Carola Kruse, editors, *High Performance Computing*, pages 297–316, Cham, 2023. Springer Nature Switzerland.
- [41] David A Lifka. The anl/ibm sp scheduling system. In *Workshop on Job Scheduling Strategies for Parallel Processing*, pages 295–303. Springer, 1995.
- [42] Aristidis Likas, Nikos Vlassis, and Jakob J Verbeek. The global k-means clustering algorithm. *Pattern recognition*, 36(2):451–461, 2003.
- [43] Andrea Macchi, Marco Borghesi, and Matteo Passoni. Ion acceleration by superintense laser-plasma interaction. *Rev. Mod. Phys.*, 85:751–793, May 2013.
- [44] Mariarosa Marchese, Federica de Trizio, Giancarlo Sciddurlo, Nicola Cordeschi, Giuseppe Piro, and Gennaro Boggia. Greentent: An intent-based framework for sustainable network orchestration. In *2026 IEEE International Conference on Communications Workshops (ICC Workshops)*, page 6, Glasgow, 2026.
- [45] Thijs Metsch, Magdalena Viktorsson, Adrian Hoban, Monica Vitali, Ravi Iyer, and Erik Elmroth. Intent-driven orchestration: Enforcing service level objectives for cloud native deployments. *SN Computer Science*, 4(3):268, 2023.
- [46] Sparsh Mittal and Jeffrey S Vetter. A survey of cpu-gpu heterogeneous computing techniques. *ACM Computing Surveys (CSUR)*, 47(4):1–35, 2015.
- [47] D Morneau. Pan-genomes: moving beyond the reference. *Nat. Res*, 2021.
- [48] Amitash Nanda, Md Kamal Hossain Chowdhury, Hannah Ross, and Kevin Gott. Exploring dynamic load balancing algorithms for block-structured mesh-and-particle simulations in amrex. In *Practice and Experience in Advanced Research Computing 2025: The Power of Collaboration*, PEARC ’25, New York, NY, USA, 2025. Association for Computing Machinery.

- [49] Tapasya Patki, Dong Ahn, Daniel Milroy, Jae-Seung Yeom, Jim Garlick, Mark Grondona, Stephen Herbein, and Thomas Scogland. Fluxion: A scalable graph-based resource model for hpc scheduling challenges. In *Proceedings of the SC'23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, pages 2077–2088, 2023.
- [50] S. J. Pennycook, J. D. Sewall, and V. W. Lee. A metric for performance portability, 2016.
- [51] Daniel Reed, Dennis Gannon, and Jack Dongarra. Reinventing high performance computing: Challenges and opportunities, 2022.
- [52] Michael E. Rowan, Kevin N. Gott, Jack Deslippe, Axel Huebl, Maxence Thévenet, Remi Lehe, and Jean-Luc Vay. In-situ assessment of device-side compute work for dynamic load balancing in a gpu-accelerated pic code. In *Proceedings of the Platform for Advanced Scientific Computing Conference*, PASC '21, New York, NY, USA, 2021. Association for Computing Machinery.
- [53] Jouni Sirén, Erik Garrison, Adam M Novak, Benedict Paten, and Richard Durbin. Haplotype-aware graph indexes. *Bioinformatics*, 36(2):400–407, 2020.
- [54] Jouni Sirén, Jean Monlong, Xian Chang, Adam M. Novak, Jordan M. Eizenga, Charles Markello, Jonas A. Sibbesen, Glenn Hickey, Pi-Chuan Chang, Andrew Carroll, Namrata Gupta, Stacey Gabriel, Thomas W. Blackwell, Aakrosh Ratan, Kent D. Taylor, Stephen S. Rich, Jerome I. Rotter, David Haussler, Erik Garrison, and Benedict Paten. Pangenomics enables genotyping of known structural variants in 5202 diverse genomes. *Science*, 374(6574):abg8871, 2021.
- [55] Joseph Skovira, Waiman Chan, Honbo Zhou, and David Lifka. The easy—loadleveler api project. In *Workshop on Job Scheduling Strategies for Parallel Processing*, pages 41–47. Springer, 1996.
- [56] Mahmoud Junior Suleman. The use of high-performance computing services in university settings: A usability case study of the university of cincinnati’s high-performance computing cluster. Master’s thesis, University of Cincinnati, 2023.
- [57] J. Sun, Z. Gao, D. Grant, K. Nawaz, P. Wang, C. M. Yang, P. Boudreaux, S. Kowalski, and S. Huff. Energy dataset of Frontier supercomputer for waste heat recovery. *Scientific Data*, 11(1):1077, October 2024.
- [58] Michela Taufer, Daniel Milroy, Todd Gamblin, Al Jones, Bill Magro, Heike Poxon, and Seetharami Seelam. Hpc and cloud convergence beyond technical boundaries: Strategies for economic sustainability, standardization, and data accessibility. *IEEE Computer*, 57(6):128–136, 2024.

- [59] Mark Taylor, Peter M Caldwell, Luca Bertagna, Conrad Clevenger, Aaron Donahue, James Foucar, Oksana Guba, Benjamin Hillman, Noel Keen, Jayesh Krishna, et al. The simple cloud-resolving e3sm atmosphere model running on the frontier exascale system. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pages 1–11, 2023.
- [60] TOP500 Project. TOP500 list – November 2025. <https://top500.org/lists/top500/2025/11/>, November 2025. 66th edition. Accessed: April 2026.
- [61] Dan Tsafir, Yoav Etsion, and Dror G Feitelson. Backfilling using system-generated predictions rather than user runtime estimates. *IEEE Transactions on Parallel and Distributed Systems*, 18(6):789–803, 2007.
- [62] U.S. Energy Information Administration. Electricity use in homes, 2025. Avg. 10,500 kWh/year per U.S. household.
- [63] Manikya Swathi Vallabhajosyula and Rajiv Ramnath. Insights from the harp framework: Using an ai-driven approach for efficient resource allocation in hpc scientific workflows. In *Practice and Experience in Advanced Research Computing 2023: Computing for the Common Good*, pages 341–344. 2023.
- [64] Manikya Swathi Vallabhajosyula and Rajiv Ramnath. Towards characterizing dnns to estimate training time using harp (hpc application resource (runtime) predictor). In *Practice and Experience in Advanced Research Computing 2023: Computing for the Common Good*, pages 483–485. 2023.
- [65] Cédric Villani et al. *Optimal transport: old and new*, volume 338. Springer, 2009.
- [66] William A. Ward, Carrie L. Mahood, and John E. West. Scheduling jobs on parallel systems using a relaxed backfill strategy. In Dror G. Feitelson, Larry Rudolph, and Uwe Schwiegelshohn, editors, *Job Scheduling Strategies for Parallel Processing*, pages 88–102, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.
- [67] WarpX Community. Laser-ion acceleration with a planar target, 2024. Accessed: May 2026.
- [68] Christin Whittton, William Jones, Craig Walker, Vanessa Job, Steven Senator, and Nathan DeBardeleben. Are we there yet? predicting the queue wait times for hpc jobs. In *2025 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 1–12, 2025.
- [69] SC Wilks, AB Langdon, Thomas E Cowan, Markus Roth, Malay Singh, Stephen Hatchett, MH Key, D Pennington, Andrew Mackinnon, and RA Snavely. Energetic proton generation in ultra-intense laser–solid interactions. *Physics of plasmas*, 8(2):542–549, 2001.

- [70] Xingfu Wu, Valerie Taylor, and Zhiling Lan. Performance and energy improvement of ecp proxy app sw4lite under various workloads. In *2021 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*, pages 17–24, 2021.
- [71] Andy B Yoo, Morris A Jette, and Mark Grondona. Slurm: Simple linux utility for resource management. In *Workshop on job scheduling strategies for parallel processing*, pages 44–60. Springer, 2003.
- [72] Weiqun Zhang, Ann Almgren, Vince Beckner, John Bell, Johannes Blaschke, Cy Chan, Marcus Day, Brian Friesen, Kevin Gott, Daniel Graves, et al. AMReX: a framework for block-structured adaptive mesh refinement. *Journal of Open Source Software*, 4(37):1370, 2019.
- [73] Weiqun Zhang, Ann Almgren, Vince Beckner, John Bell, Johannes Blaschke, Cy Chan, Marcus Day, Brian Friesen, Kevin Gott, Daniel Graves, Max Katz, Andrew Myers, Tan Nguyen, Andrew Nonaka, Michele Rosso, Samuel Williams, and Michael Zingale. AMReX: a framework for block-structured adaptive mesh refinement. *Journal of Open Source Software*, 4(37):1370, May 2019.

Appendix A

miniGiraffe information

Table A.1: Hardware platforms used for evaluation

	local-intel	local-amd	chi-arm	chi-intel
Vendor	Intel	AMD	Cavium	Intel
Processor	Xeon 8260	EPYC 9554	ThunderX2 99xx	Xeon 8380
Sockets	2	1	2	2
Frequency	2.4 GHz	3.1 GHz	2.5 GHz	2.3 GHz
Cores / socket	24	64	32	40
L3 cache / socket	35.75 MB	256 MB	64 MB	60 MB
L2 cache / core	1 MB	1 MB	256 KB	1.25 MB
L1I cache / core	32 KB	32 KB	32 KB	32 KB
L1D cache / core	32 KB	32 KB	32 KB	48 KB
Threads / core	2	2	1	2
DRAM	768 GB	768 GB	256 GB	256 GB
OS	Ubuntu 22.04.4	Debian 12	Ubuntu 22.04.5	Ubuntu 24.04.2
Compiler	gcc 11.4.0	gcc 12.2.0	gcc 11.4.0	gcc 13.3.0